

Problem Solving Class

1. Write a program to Find the greatest among two numbers.
2. Write a program to Find the greatest among three numbers.
3. Write a program to Check whether a given number is positive or negative.
4. Write a program to Check whether a number is even or odd.
5. Write a program to Check whether a person is eligible to vote or not.
6. Write a program to Check whether a given year is a leap year or not.
7. Write a program to Check the given digit is present in given number or not n=125464.
8. Write a program to Check whether a given character is lowercase, uppercase, digit, or other.
9. Write a program to Sort Three Numbers in Descending Order.
10. Write a program to Check the given character are present in string or not.

Problem Solving class

11. Write a program to print number from 1 to 100.
12. Write a program to print the all even number from 1 to 100.
13. Write a program to find the sum of natural number
14. Write a program to find the sum of all even number from the given number
15. Write a program to print the number that is divisible by 3 and 5
16. Write a program to print the number that is divisible by 5 or 7
17. Write a program to print the multiplication table of given number
18. Write a program to find the factorial of given number
19. Write a program to factor of number given number
20. Write a program to find sum, product and count of factor of a given number

Problem Solving class

21. Write a program to find the sum, Count & product of a given digit Example= 46425
22. Write a program to check the last digit is even or odd Example=545785
23. Write a program to **check whether a number is a perfect square or not.**
24. Write a program to find the factorial of given number.
25. Write a program to count the factor of given number.
26. Write a program to find the factor of given number.
27. Write a program to Reverse the number
28. Write a program to Reverse the string
29. Write a program to count no of character present in your name
30. Write a program to count & print the all vowels present in your name

Problem Solving class

31. Write a program to find the Fibonacci series.
32. Write a program to Simple authentication system (login successful / failed).
33. Write a program to print index value and character present in given string
34. write a program to display the alternat even number.
35. write a program to display the alternat even number is divisible by 4.
36. write a program to maximum digit form a given number.
37. write a **program to swape** the two number
38. write a program to Check whether a number is a strong number or not.
➔ write a program to Print all strong number within a given range (1 to 1000)
39. write a program to Find the square and cube of each number from 1 to 10.
- 40. Write a program to check whether a given number is perfect or not**
➔ write a program to Print all perfect numbers within a given range (1 to 1000).

Problem Solving Class

41. write a program to Check whether a given number is a palindrome or not.
42. write a program to print the all-palindrome number from 1 to 1000
43. write a program to Check whether a given number is an Armstrong number or not.
44. write a program to find the all-Armstrong number from 1 to 1000hnm .
45. write a program to Check whether a given string is a palindrome (e.g., madam).
46. Write a program to find **all palindrome string from given paragraph**.
47. Write a program to find the area of triangle
48. Write a program to find the area of Square
49. Write a program to Print numbers whose digits are in increasing order.
50. Write a program to find the **smallest digit** in a number.

Problem Solving Class

51. Write a program to check the given number is prime or not
52. Write a program to print all prime number in interval
53. Write a program to count and sum of all prime number from 50 to 500
54. Write a program to find the nth prime number
55. Write a program to find the prime factor of a given number
56. Write a program to check the given number is co-prime or not
57. write a program to display nearest prime number for given number.
58. write a program to check the nearest prime number is palindrome or not.
59. write a program to Find the GCD and LCM of two given numbers.
60. Write a program to **find the largest prime number** within a given range.

Problem Solving class

61. Write a program to check whether a number is a power of 2 or not.
62. Write a program to count the number of digits in a given number.
63. Write a program to find the sum of first N natural numbers.
64. Write a program to find the sum of digits of a number until it becomes a single digit.
65. Write a program to check whether a number is a harshad (Niven) number or not.
66. Write a program to print all harshad numbers from 1 to 100.
67. Write a program to find the largest digit in a given number.
68. Write a program to check whether two numbers are equal without using comparison operators.
69. Write a program to convert a decimal number to binary.
70. Write a program to convert a binary number to decimal.

Problem Solving

81. Write a program to print the first N prime numbers.
82. Write a program to print numbers that are divisible by both 2 and 3 from 1 to 100.
83. Write a program to find the sum of odd digits in a number.
84. Write a program to check whether a number is a Spy number or not.
85. Write a program to print all Spy numbers between 1 and 100.
86. Write a program to count how many even and odd digits are present in a number.
87. Write a program to check whether a number is a Duck number or not.
88. Write a program to print all Duck numbers from 1 to 100.
89. Write a program to find the difference between the sum of even and odd digits.
90. Write a program to print numbers whose sum of digits is even.

Problem Solving

91. Write a program to count uppercase and lowercase letters in a string.
92. Write a program to remove vowels from a given string.
93. Write a program to check whether two strings are anagrams or not.
94. Write a program to find the frequency of each character in a string.
95. Write a program to replace all spaces in a string with hyphens (-).
96. Write a program to reverse each word in a given string.
97. Write a program to find the longest word in a string.
98. Write a program to check whether a string contains only digits.
99. Write a program to count the number of words in a sentence.
100. Write a program to capitalize the first letter of each word in a string.

Problem Solving – Advanced Logical Thinking

101. Write a program to check whether a number is an Automorphic number or not.
102. Write a program to print all Automorphic numbers from 1 to 1000.
103. Write a program to find the sum of prime digits in a number.
104. Write a program to check whether a number is a Disarium number or not.
105. Write a program to print all Disarium numbers from 1 to 100.
106. Write a program to check whether a number is a Peterson number or not.
107. Write a program to print all Peterson numbers from 1 to 500.
108. Write a program to generate Pascal's Triangle.
109. Write a program to find the missing number in a given sequence.
110. Write a program to check whether a number is a Fibonacci number or not.

LIST-BASED PROBLEM

- 1) write a python program to find the min and max value from a given list.
- 2) write a python program to rotate a shift k element from left to right from the given list.
- 3) write a python program to display unique vowels present in the given word
- 4) write a python program to search the particular elements without using pre-defined function.
- 5) write a python program to remove the duplicate elements from a list.
- 6) write a python program to display the how many times given elements is repeated in a given list.
- 7) write a python program to find the maximum number of repeated elements is given list
- 8) write a python program to perform the slicing operation given list.
- 9) write a python program to shift 1st k element from left to write
- 10) sort the first k element ascending order remaining n-k elements in descending order
- 11) write a python program to shift the all zero in end of the list.
- 12) write a python program to find the element which is repeating exactly one time which is given list within the given list all the elements will repeated more than one times except one element
- 13) write a python program to perform Bauble sort.
- 14) write a python program to perform the selection sort.
- 15) write a python program to take all input are 2-d list in same row are separated will Coolum are separated with space.
- 16) write a python program to sort the first k-element ascending order and reaming n-k element in descending order.
- 17) write a python program to shift all the zeros to end of the list.
- 18) Write a python program to find the element which is repeating exactly one time which is given list within the given list all the elements will repeated more than one times except one element
- 19) Write a python program to find the maximum number of 2-d list:

- 20) write a python program to find the maximum number list inside list.
- 21) write a python program to reverse the given list without using pre-defined function.
- 22) write a python program to merge two lists into a single list.
- 23) write a python program to find the second largest element from the given list.
- 24) write a python program to find the second smallest element from the given list.
- 25) write a python program to count the even and odd numbers present in the list.
- 26) write a python program to separate positive and negative numbers from the given list.
- 27) write a python program to find the sum of all elements in the given list.
- 28) write a python program to find the average of elements present in the list.
- 29) write a python program to insert an element at a particular position in the list.
- 30) write a python program to delete an element from a particular position in the list.
- 31) write a python program to reverse each element present in the given list of strings.
- 32) write a python program to check whether the given list is palindrome or not.
- 33) write a python program to find the common elements between two lists.
- 34) write a python program to find the missing number from the given list.
- 35) write a python program to convert all negative numbers into positive numbers in the list.
- 36) write a python program to create a new list with square of all elements from the given list.
- 37) write a python program to find the frequency of each element in the given list.
- 38) write a python program to swap the first and last element of the given list.
- 39) write a python program to split the list into two equal parts.
- 40) write a python program to find the largest element in each row of a 2-D list.

STRING BASED PROBLEM

1. Write a program to reverse the given string.
2. Write a program to reverse the order of the words
3. Write a program to check the given string is palindrome or not
4. Write a program to reverse the internal content of each word
5. Write a program to reverse the string without reversing the special symbols
6. Write a program to print characters at odd position and even position for the given string
7. Write a program to extract only numeric character from a largest possible even number first that given string are contains alpha number characters.
8. Write a program to merge characters of 2 string into a single string by taking characters alternatively.
9. Write a program to return the super reduced string.
10. Write a program to sort the characters of the string and first alphabets symbols followed by numeric values.
11. Write a program to check where given string is pangram or not
12. Write a program to check where given string is anagram or not
13. Write a program for the following requirement
Input: a3b6c2
Output: aaabbbbbbbcc
14. Write a program to perform the following activity
Input: a4k3b2
Output: aeknbd
15. Write a program to Remove Duplicate characters from the given input string?
Input: ABCDABACBEEDBCFDBDCCFCFDFC
Output: ABCDEF

16. Write a program to find the number of occurrences of each character present in the given string
17. Write a program to perform the following task?
Input: 'one two three four five six seven'
Output: 'one owt three ruof five xis seven'
18. Write a program to find the character weight of alphabet of given string s "TWKSAA"
19. Write a program to perform the following task
Input: aaabbcdcdcde
Output: [('a',4), ('b',2), ('c',1), ('d', 3), ('e',1)]
a4b2c1d3e1
20. Write a program to get opposite of previous (above) question.
21. Write a program to check the how many times 'rama' can be form a given string.
Input:rrmmaaarwxyz
22. Write a program to display the edit distance to make given string as a palindrome. Edit distance between a to c is 2 i.e, a-b then b-c , a to z is 25
23. Reverse the Given String
24. Reverse the Order of Words in a String
25. Check Whether the Given String is Palindrome or Not
26. Reverse the Internal Content of Each Word
27. Reverse the String Without Reversing Special Symbols
28. Print Characters at Odd Position and Even Position
29. Extract Numeric Characters and Form Largest Possible Even Number
30. Merge Characters of Two Strings Alternatively
31. Return the Super Reduced String
32. Sort Characters of String with Alphabets First and Numbers Later
33. Check Whether Given String is Pangram or Not
34. Check Whether Given String is Anagram or Not
35. Expand String Based on Given Number
36. Perform Character Shifting Activity Using ASCII Values
37. Remove Duplicate Characters from the Given String
38. Find the Number of Occurrences of Each Character
39. Reverse Alternate Words in the Sentence
40. Find the Character Weight of Given String
41. Count Consecutive Characters in String
42. Expand the Compressed String
43. Check How Many Times "rama" Can Be Formed
44. Find Edit Distance to Make String Palindrome
45. Count Total Number of Vowels in the Given String
46. Count Total Number of Consonants in the Given String
47. Find the First Non-Repeated Character in the Given String
48. Replace Spaces with Hyphen in the Given String
49. Count Uppercase and Lowercase Characters
50. Remove All Spaces from the Given String
51. Convert First Character of Each Word into Uppercase
52. Find the Longest Word in the Given Sentence
53. Find the Smallest Word in the Given Sentence
54. Check Whether Two Strings are Rotations of Each Other

55. Find Duplicate Characters in the Given String
56. Convert Lowercase Characters into Uppercase
57. Convert Uppercase Characters into Lowercase
58. Check Whether String Contains Only Digits
59. Find ASCII Value of Each Character in the String
60. Count Total Number of Words in the Given Sentence
61. Find Total Number of Digits Present in the String
62. Find Total Number of Special Characters in the String
63. Remove All Vowels from the Given String
64. Check Whether the String Starts with a Vowel
65. Print Each Character with Its Position
66. Find the Frequency of Vowels in the Given String
67. Reverse Only Words Having Even Length
68. Find the Highest Frequency Character in the String
69. Replace All Digits in the String with '*'
70. Find the Last Non-Repeated Character in the Given String
71. Check Whether Two Strings are Equal Ignoring Case
72. Find the Percentage of Uppercase, Lowercase, Digits, and Special Characters

Function Question

Simple Questions (1–10)

1. Write a function to add two numbers.
2. Write a function that returns the square of a number.
3. Create a function that checks if a number is even or odd.
4. Write a function to find the maximum of three numbers.
5. Define a function to calculate the factorial of a number.
6. Create a function that prints the multiplication table of a number.
7. Write a function to calculate the area of a circle (use $\pi = 3.14$).
8. Define a function that returns the sum of all numbers from 1 to n.
9. Write a function that returns the average of three numbers.
10. Create a function that returns the cube of a number.

Medium-Level Questions (11–20)

11. Write a function to check if a number is prime.
12. Create a function that finds the sum of digits of a number.
13. Write a function that returns the reverse of a number.
14. Define a function that checks if a number is a palindrome.
15. Write a function to find the greatest common divisor (GCD) of two numbers.
16. Create a function that counts the number of digits in a number.
17. Write a function that returns the sum of even numbers in a given range.
18. Define a function to print all prime numbers between 1 and n.
19. Create a function that converts Celsius to Fahrenheit.
20. Write a function that accepts marks of 5 subjects and returns the percentage.

High-Level Function-Based Questions

21. Write a function to find the nth Fibonacci number using recursion.
22. Create a function to check if a number is an Armstrong number (e.g., $153 = 1^3 + 5^3 + 3^3$).
23. Write a function that returns all factors of a given number.

24. Create a function that takes a number and returns True if it is a perfect number.
(A perfect number is equal to the sum of its proper divisors, e.g., $28 = 1 + 2 + 4 + 7 + 14$)
25. Define a function that finds the least common multiple (LCM) of two numbers.
26. Write a function that accepts a list of numbers and returns the second largest number.
27. Create a function that returns the sum of all prime numbers between 1 and n.
28. Write a function that returns True if the given number is a Harshad number.
(A number divisible by the sum of its digits)
29. Create a function that calculates and returns the digital root of a number.
(Repeat summing digits until a single digit is left. Example: $987 \rightarrow 9+8+7=24 \rightarrow 2+4=6$)
30. Write a function that returns the number of trailing zeros in the factorial of a number.

Solution

1. Write a Program to check Greatest Among Two Numbers

Program

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
if a > b:
    print("Greatest number is:", a)
else:
    print("Greatest number is:", b)
```

Sample Input: 5 9

Output: Greatest: 9

Logic Explanation: Greatest Among Two Numbers

Program Logic:

- Two numbers are taken as input and stored in variables a and b.
- The if condition checks whether a is greater than b.
- If the condition is true, a is printed as the greatest number.
- Otherwise, b is printed.

Mathematical Logic:

- Between two numbers, the one with the **higher value** is the greatest.
- The comparison operator (>) is used to compare the values of a and b.
- Based on the comparison result, the larger number is selected.

2. Write a Program to check Greatest Among Three Numbers

Program

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))
if a >= b and a >= c:
    print("Greatest number is:", a)
elif b >= a and b >= c:
    print("Greatest number is:", b)
else:
    print("Greatest number is:", c)
```

Sample Input: 4 7 2

Output: Greatest number is: 7

Logic Explanation: Greatest Among Three Numbers

Program Logic:

- Three numbers are taken as input and stored in variables a, b, and c.
- The first condition checks whether a is greater than or equal to both b and c.
- If the condition is true, a is printed as the greatest number.
- Otherwise, the second condition checks whether b is greater than or equal to both a and c.
- If this condition is true, b is printed.
- If none of the above conditions are true, c is printed as the greatest number.

Mathematical Logic:

- The greatest number must be greater than or equal to the other two numbers.
- Logical AND (and) ensures comparison with both values.
- The comparison operator ($\geq$) allows correct selection even if numbers are equal.

3. Write a Program to check Positive or Negative Number

```
n = int(input("Enter number: "))
if n >= 0:
    print("Positive")
else:
    print("Negative")
```

Sample Input: -5

Output: Negative

Logic Explanation: Positive or Negative Number

Program Logic:

- A number is taken as input and stored in variable n.
- The condition checks whether n is greater than or equal to 0.
- If the condition is true, the number is printed as Positive.
- Otherwise, the number is printed as Negative.

Mathematical Logic:

- Numbers greater than or equal to zero are considered positive.
- Numbers less than zero are considered negative.
- The comparison operator ($\geq$) is used to apply this rule.

4. Write a Program to check Even or Odd Number

Program

```
n = int(input("Enter number: "))
if n % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Sample Input: 10

Output: Even

Logic Explanation: Even or Odd Number

Program Logic:

- A number is taken as input and stored in variable n.
- The number is divided by 2 using the modulus operator (%).
- The condition checks whether the remainder is equal to 0.
- If the remainder is 0, the number is printed as Even.
- Otherwise, the number is printed as Odd.

Mathematical Logic:

- Even numbers are exactly divisible by 2 (remainder = 0).
- Odd numbers leave a remainder of 1 when divided by 2.
- The modulus operator is used to determine the remainder.

5. Write a Program to check Voting Eligibility

Program

```
age = int(input("Enter age: "))
if age >= 18:
    print("Eligible to vote")
else:
    print("Not eligible to vote")
```

Sample Input: 20

Output: Eligible to vote

Logic Explanation: Voting Eligibility

Program Logic:

- Age is taken as input and stored in variable age.
- The condition checks whether age is greater than or equal to 18.
- If the condition is true, the person is printed as Eligible to vote.
- Otherwise, the person is printed as Not eligible to vote.

Mathematical / Rule Logic:

- According to voting rules, the minimum age required to vote is 18 years.
- The comparison operator ($\geq$) is used to apply this rule.

6. Write a Program to check Leap Year Check

Program

```
year = int(input("Enter year: "))
if (year % 400 == 0) or (year % 4 == 0 and year % 100 != 0):
    print("Leap Year")
else:
    print("Not Leap Year")
```

Sample Input: 2024

Output: Leap Year

Logic Explanation: Leap Year Check

Program Logic:

- A year is taken as input and stored in variable year.
- The program checks divisibility conditions using the modulus operator (%).
- If the year is divisible by 400, it is a Leap Year.
- Otherwise, if the year is divisible by 4 but not divisible by 100, it is also a Leap Year.
- If none of these conditions are satisfied, it is printed as Not Leap Year.

Mathematical Logic:

- A leap year contains 366 days instead of 365.
- Leap year rules:
- Years divisible by 400 are leap years.
- Years divisible by 4 are leap years unless they are divisible by 100.
- Divisibility is checked using the modulus operator.

7. Write a Program to check given Digit Present in Number or not

Program

```

n = 125464
d = input("Enter digit: ")
if d in str(n):
    print("Digit Present")
else:
    print("Digit Not Present")

```

Sample Input: 5

Output: Digit Present

Logic Explanation: Digit Check in Number

Program Logic:

- The number n is given as 125464.
- A digit is taken as input from the user.
- The number is converted into a string using str(n).
- The membership operator (in) checks whether the entered digit exists in the number.
- If found, the program prints Digit Present; otherwise, Digit Not Present.

Mathematical Logic:

- Any number is composed of digits from 0 to 9.
- Checking digit presence means verifying whether the digit matches any digit in number.
- This is logically equivalent to scanning each digit of the number.

8. Write a Program to check given character is upper case, lowercase, digit, other

Program

```

ch = input("Enter character: ")
if ch.isupper():
    print("Uppercase Letter")
elif ch.islower():
    print("Lowercase Letter")
elif ch.isdigit():
    print("Digit")
else:
    print("Special Character")

```

```

ch = input("Enter character: ")
if ch >= 'A' and ch <= 'Z':
    print("Uppercase Letter")
elif ch >= 'a' and ch <= 'z':
    print("Lowercase Letter")
elif ch >= '0' and ch <= '9':
    print("Digit")
else:
    print("Special Character")

```

Sample Input: @

Output: Special Character

Logic Explanation: Character Type Check

Program Logic:

- A single character is taken as input and stored in variable ch.
- The first condition checks whether the character lies between 'A' and 'Z'.
- If true, it is an Uppercase letter.
- The next condition checks whether the character lies between 'a' and 'z'.
- If true, it is a Lowercase letter.
- The next condition checks whether the character lies between '0' and '9'.
- If true, it is a Digit.
- If none of these conditions are true, it is treated as a Special Character.

Mathematical / ASCII Logic:

- Each character has an ASCII value.
- Uppercase letters (A–Z), lowercase letters (a–z), and digits (0–9) occupy fixed ASCII ranges.
- Comparing characters using relational operators (>=, <=) checks these ranges.

9. Write a program to Sort Three Numbers in Descending Order

Program

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))
if a >= b and a >= c:
    if b >= c:
        print(a, b, c)
    else:
        print(a, c, b)
elif b >= a and b >= c:
    if a >= c:
        print(b, a, c)
    else:
        print(b, c, a)
else:
    if a >= b:
        print(c, a, b)
    else:
        print(c, b, a)
```

Sample Input: 3 1 2

Output: 3 2 1

Logic Explanation: Sorting Three Numbers in Descending Order

Program Logic:

- Three numbers are taken as input and stored in variables a, b, and c.
- The first outer condition checks which number is the largest.
- Once largest number is identified, nested conditions compare the remaining two numbers.
- Based on comparisons, numbers are printed from largest to smallest.
- No swapping of variables is done — only conditional checks.

Mathematical Logic:

- Descending order means arranging numbers from highest to lowest.
- Each number is compared using relational operators ($\geq$).
- Logical AND ensures correct identification of the largest number.

10. write the program to check the given Character is Present in String or not

Program

```
s = input("Enter string: ")
ch = input("Enter character: ")

if ch in s:
    print("Character Present")
else:
    print("Character Not Present")
```

Sample Input:

Enter string: Python
Enter character: y

Output: Character Present

Logic Explanation: Character Present in String

Program Logic:

- A string is taken as input and stored in variable s.
- A character is taken as input and stored in variable ch.
- The in operator is used to check whether the character exists in the string.
- If the condition is true, Character Present is printed.
- Otherwise, Character Not Present is printed.

Mathematical / Logical Concept:

- A string is a collection of characters.
- The membership operator (in) checks whether a character belongs to a string.
- If the character exists in the string, the condition becomes true.
- If it does not exist, the condition becomes false.

11. Write a program to Print the Numbers from 1 to 100

Program

```
for i in range(1, 101):
    print(i)
```

Sample Output: 1 2 3 ... 100

Logic Explanation: Print Numbers from 1 to 100

Program Logic:

- A for loop is used to repeat the task.
- The range(1, 101) function generates numbers starting from 1 up to 100.
- Each number generated by the loop is stored in variable i.
- The print() function displays the value of i one by one.

Mathematical Logic:

- Counting starts from 1.
- Each next number is obtained by adding 1 to the previous number.
- The process continues until the number reaches 100.
- This follows a simple sequence of natural numbers from 1 to 100.

12. Write a program to Print All Even Numbers from 1 to 100

Program

```
for i in range(1, 101):
    if i % 2 == 0:
        print(i)
```

Sample Output: 2 4 6 ... 100

Logic Explanation: Print All Even Numbers from 1 to 100

Program Logic:

- A for loop runs from 1 to 100 using range(1, 101).
- Each number is stored in variable i.
- The condition i % 2 == 0 checks whether the number is even.
- If the condition is true, the number is printed.

Mathematical Logic:

- Even numbers are those which are completely divisible by 2.
- When a number is divided by 2 and the remainder is 0, it is an even number.
- The modulo operator (%) is used to find the remainder.

13. Write a program to find the Sum of Natural Numbers

Program

```
n = int(input("Enter number: "))
s = 0
```

```

for i in range(1, n+1):
    s += i
print("Sum =", s)

```

Sample Input: 5

Output: Sum = 15

Program Logic:

- A number is taken as input and stored in variable n.
- A variable s is initialized to 0 to store the sum.
- A for loop runs from 1 to n.
- Each value of i is added to s.
- After the loop ends, the final sum is printed.

Mathematical Logic:

- Natural numbers start from 1 and increase by 1.
- The sum of first n natural numbers is given by the formula:

$$n(n + 1) / 2$$
- The loop-based approach adds each number one by one to get the same result.

14. Write a program to find Sum of All Even Numbers up to Given Number

Program

```

n = int(input("Enter number: "))
s = 0
for i in range(1, n+1):
    if i % 2 == 0:
        s += i
print ("Sum of even numbers =", s)

```

Sample Input: 10

Output: 30

Logic: Add only numbers divisible by 2.

Program Logic:

- A number is taken as input and stored in variable n.
- A variable s is initialized to 0 to store the sum.
- A for loop runs from 1 to n.
- Each number is checked using the condition $i \% 2 == 0$.
- If the condition is true, the number is added to the sum.
- After the loop completes, the total sum of even numbers is printed.

Mathematical Logic:

- Even numbers are those which are divisible by 2.
- Only numbers with remainder 0 when divided by 2 are added.
- This gives the sum of all even numbers up to the given number.

15. Write a program to find all Numbers Divisible by 3 and 5 from 1 to 100

Program

```

for i in range (1, 101):
    if i % 3 == 0 and i % 5 == 0:
        print(i)

```

Output: 15, 30, 45, 60, 75, 90

Logic Explanation: Numbers Divisible by 3 and 5

Program Logic:

- A for loop runs from 1 to 100.
- Each number is stored in variable i.

- The condition $i \% 3 == 0$ and $i \% 5 == 0$ checks divisibility by both numbers.
- If the condition is true, the number is printed.

Mathematical Logic:

- A number divisible by both 3 and 5 must be divisible by their LCM.
- The LCM of 3 and 5 is 15.
- Therefore, numbers divisible by 15 satisfy the condition.

16. Write program to find the all Numbers Divisible by 5 or 7 from 1 to 100

Program

```
for i in range(1, 101):
    if i % 5 == 0 or i % 7 == 0:
        print(i)
```

Logic Explanation: Numbers Divisible by 5 or 7

Program Logic:

- A for loop runs from 1 to 100.
- Each number is stored in variable i.
- The condition $i \% 5 == 0$ or $i \% 7 == 0$ checks divisibility.
- If the number is divisible by 5 or 7, it is printed.

Mathematical Logic:

- A number divisible by 5 has remainder 0 when divided by 5.
- A number divisible by 7 has remainder 0 when divided by 7.
- The logical OR (or) operator returns true if any one condition is satisfied.
- Hence, numbers divisible by either 5 or 7 are displayed.

17. Write a Program to Print the Multiplication Table of a Number

Program:

```
n = int(input("Enter number: "))
for i in range(1, 11):
    print(n, "x", i, "=", n*i)
```

Sample Input:

5

Sample Output:

```
5 x 1 = 5
5 x 2 = 10
...
5 x 10 = 50
```

Logic Explanation: Multiplication Table

Program Logic:

- A number is taken as input and stored in variable n.
- A for loop runs from 1 to 10.
- In each iteration, n is multiplied by i.
- The result is printed in table format.

Mathematical Logic:

- Multiplication table is obtained by multiplying the number with values from 1 to 10.
- This produces successive multiples of the given number.

18. Write a Program to Find the Factorial of a Number

Program:

```
n = int(input("Enter number: "))
fact = 1
for i in range(1, n+1):
```

```
fact *= i
print("Factorial =", fact)
```

Sample Input: 5

Output: Factorial = 120

Logic Explanation: Factorial of a Number

Program Logic:

- A number is taken as input and stored in variable n.
- A variable fact is initialized to 1.
- A loop runs from 1 to n.
- Each number is multiplied with fact.
- Final factorial value is printed.

Mathematical Logic:

- Factorial of a number is defined as:

$$n \times (n - 1) \times (n - 2) \times \dots \times 1$$

- Example: $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$.

19. Write a Program to Find the Factors of a Number

Program:

```
n = int(input("Enter number: "))
for i in range(1, n+1):
    if n % i == 0:
        print(i)
```

Sample Input: 12

Output: 1 2 3 4 6 12

Logic Explanation: Factors of a Number

Program Logic:

- A number is taken as input and stored in variable n.
- A loop runs from 1 to n.
- Each number is checked using $n \% i == 0$.
- If remainder is 0, the number is a factor and printed.

Mathematical Logic:

- A factor divides a number completely without leaving a remainder.
- All such numbers are called factors of the given number.

20. Write a Program to Find the Sum, Product, and Count of Factors of a Number

Program:

```
n = int(input("Enter number: "))
s = 0
p = 1
count = 0
for i in range(1, n+1):
    if n % i == 0:
        s += i
        p *= i
        count += 1
print("Sum =", s)
print("Product =", p)
print("Count =", count)
```

Sample Input: 6

Output:

Sum = 12
Product = 36
Count = 4

Logic Explanation: Sum, Product and Count of Factors**Program Logic:**

- A number is taken as input and stored in variable n.
- Variables s, p, and count are initialized.
- Loop runs from 1 to n.
- If a number is a factor, it is added to sum, multiplied to product, and counted.
- Final values are printed.

Mathematical Logic:

- Factors are numbers that divide a given number exactly.
- All factors are used to calculate their sum, product, and total count.
- Example: Factors of 6 → 1, 2, 3, 6.

21. Write a program to find the Sum, Count & Product of Digits**Program**

```
num = int(input("Enter a number: "))
temp = num
sum_d = 0
count = 0
product = 1
while temp > 0:
    digit = temp % 10
    sum_d += digit
    product *= digit
    count += 1
    temp //= 10
print("Sum of digits:", sum_d)
print("Count of digits:", count)
print("Product of digits:", product)
```

Sample Input: 46425

Output: Sum=21, Count=5, Product=960

Logic Explanation**Program Logic:**

- A number is taken as input from the user.
- A loop runs until the number becomes 0.
- Each digit is extracted using num % 10.
- The digit is added to sum, multiplied to product, and counted.
- The number is reduced using num // 10.

Mathematical Logic:

- Any number can be broken into digits using modulus (%) and division (/).
- Sum = addition of all digits.
- Product = multiplication of all digits.
- Count = total digits present.

22. Write a Program to Check Whether the Last Digit is Even or Odd

Program:

```
num = int(input("Enter a number: "))
last = num % 10
if last % 2 == 0:
    print("Last digit is Even")
else:
    print("Last digit is Odd")
```

Sample Input: 545785**Output:** Last digit is Odd**Logic Explanation: Last Digit Even or Odd****Program Logic:**

- A number is taken as input and stored in variable num.
- The last digit is obtained using the modulo operator % 10.
- The last digit is checked for divisibility by 2.
- If divisible, it is even; otherwise, it is odd.

Mathematical Logic:

- Even numbers are divisible by 2.
- Odd numbers leave a remainder of 1 when divided by 2.
- Checking the last digit is sufficient to determine even or odd.

23. Write a Program to Check Whether a Given Number is a Perfect Square**Program:**

```
import math
num = int(input("Enter a number: "))
root = int(math.sqrt(num))
if root * root == num:
    print("Perfect Square")
else:
    print("Not a Perfect Square")
```

Sample Input: 25**Output:** Perfect Square**Logic Explanation: Perfect Square Check****Program Logic:**

- A number is taken as input and stored in variable num.
- Square root of the number is calculated using math.sqrt().
- The square of the integer root is compared with the original number.
- If both are equal, the number is a perfect square.

Mathematical Logic:

- A perfect square is a number that can be written as n^2 .
- Example: 1, 4, 9, 16, 25 are perfect squares.

24. Write a Program to Find the Factorial of a Given Number**Program:**

```
num = int(input("Enter a number: "))
fact = 1
for i in range(1, num+1):
    fact *= i
print("Factorial:", fact)
```

Sample Input: 5**Output:** Factorial: 120

Logic Explanation: Factorial of a Number**Program Logic:**

- A number is taken as input and stored in variable num.
- Variable fact is initialized to 1.
- Loop runs from 1 to the given number.
- Each number is multiplied to calculate factorial.

Mathematical Logic:

- Factorial of a number n is defined as:

$$n \times (n - 1) \times (n - 2) \times \dots \times 1$$

25. Write a Program to Count the Factors of a Given Number**Program:**

```
num = int(input("Enter a number: "))
count = 0
for i in range(1, num+1):
    if num % i == 0:
        count += 1
print("Number of factors:", count)
```

Sample Input: 12

Output: Number of factors: 6

Logic Explanation: Count Factors of a Number**Program Logic:**

- A number is taken as input and stored in variable num.
- A loop runs from 1 to the given number.
- If a number divides the given number completely, the counter increases.
- Final count of factors is printed.

Mathematical Logic:

- A factor is a number that divides another number exactly without remainder.
- Counting all such numbers gives the total number of factors.

26. Write a Program to Find All Factors of a Given Number**Program:**

```
num = int(input("Enter a number: "))
for i in range(1, num+1):
    if num % i == 0:
        print(i, end=" ")
```

Sample Input: 12

Output: 1 2 3 4 6 12

Logic Explanation: All Factors of a Number**Program Logic:**

- A number is taken as input and stored in variable num.
- A loop runs from 1 to the given number.
- Each number is checked using $\text{num} \% i == 0$.
- If the remainder is zero, the number is printed as a factor.

Mathematical Logic:

- Factors are numbers that divide a given number exactly.
- They leave no remainder after division.
- All such divisors are called factors of the number.

27. Write a Program to Reverse a Given Number**Program:**

```

num = int(input("Enter a number: "))
rev = 0
while num > 0:
    digit = num % 10
    rev = rev * 10 + digit
    num //= 10
print("Reversed number:", rev)

```

Sample Input: 12345

Output: Reversed number: 54321

Logic Explanation: Reverse a Number

Program Logic:

- A number is taken as input and stored in variable num.
- Variable rev is initialized to 0.
- The last digit is extracted using % 10.
- The digit is added to rev after shifting its place value.
- The original number is reduced using integer division (/).
- The process continues until the number becomes 0.

Mathematical Logic:

- When a number is multiplied by 10, its digits shift left by one place.
- Adding the extracted digit forms the reversed number.
- This process reconstructs the number in reverse order.

28. Write a program to Reverse a given String

Program-1

```

text = input("Enter a string: ")
print("Reversed string:", text[::-1])

```

program-2:

```

text = input("Enter a string: ")
rev = ""
for ch in text:
    rev = ch + rev
print("Reversed string:", rev)

```

Logic Explanation: Reverse a String

Program Logic:

- A string is taken as input from the user.
- An empty string variable rev is created to store the reversed string.
- A loop runs through each character of the original string.
- Each new character is placed in front of the previous characters (rev = ch + rev).
- This process continues until all characters are processed.
- The final string stored in rev is the reversed string.

Mathematical Logic:

- A string is a sequence of characters.
- Reversing means changing the order from first-to-last into last-to-first.
- By adding each new character to the front, positions of characters are shifted right, producing the reverse order.

29. Write a program to Count Characters in given Name

Program-1

```

name = input("Enter your name: ")
print("Total characters:", len(name))

```

Program-2:

```
name = input("Enter your name: ")
count = 0
for ch in name:
    count += 1
print("Total characters:", count)
```

Logic Explanation: Count Characters in Name

Program Logic:

- The user enters a name (string input).
- A variable count is initialized to 0.
- A loop runs through each character in the string.
- For every character, the value of count increases by 1.
- After the loop ends, count holds the total number of characters.

Mathematical Logic:

- A string is a sequence of characters.
- Counting means increasing a value by 1 for each element present.
- Total characters = number of times the loop runs.

30. Write a program to Count & Print Vowels present in your name

Program-1

```
name = input("Enter your name: ").lower()
vowels = "aeiou"
count = 0
for ch in name:
    if ch in vowels:
        print(ch, end=" ")
        count += 1
print("\nTotal vowels:", count)
```

Program-2

```
name = input("Enter your name: ").lower()
count = 0
for ch in name:
    if ch == 'a' or ch == 'e' or ch == 'i' or ch == 'o' or ch == 'u':
        print(ch, end=" ")
        count += 1
print("\nTotal vowels:", count)
```

Program Logic:

- The user enters a name. .lower() converts all characters to lowercase for easy comparison.
- A variable count is set to 0. A loop checks each character one by one.
- The if condition uses OR (or) to compare the character with each vowel (a, e, i, o, u).
- If the condition is true, the vowel is printed and count increases.
- After the loop, total vowels are displayed.

Mathematical Logic:

- Vowels are specific letters: a, e, i, o, u.
- Logical OR means if any one condition is true, the result is true.
- Counting increases by 1 for each vowel found.

31. Write a program to find the Fibonacci series.

Program

```
n = int(input("Enter number"))
```

```
n = int(input("Enter number of terms: "))
a = -1
b = 1
for i in range(n):
    c = a + b
    print(c, end=" ")
    a = b
    b = c
```

```

a = 0
b = 1
print(a, b, end=" ")
for i in range(3, n+1):
    c = a + b
    print(c, end=" ")
    a = b
    b = c

```

Logic Explanation: Fibonacci Series

Program Logic:

- The user enters how many terms of Fibonacci series to print.
- Two variables are initialized:
→ a = -1
→ b = 1
- A loop runs n times.
- Each time, next number is calculated as $c = a + b$.
- The value of c is printed.
- Then values are shifted:
→ a becomes old b
→ b becomes new c

Mathematical Logic:

- Fibonacci series rule:
Next term = Sum of previous two terms
- Starting trick:
-1 + 1 = 0
1 + 0 = 1
0 + 1 = 1
1 + 1 = 2
1 + 2 = 3 ...
So the series becomes:
0, 1, 1, 2, 3, 5, 8, ...

32. Write a program to Simple authentication system (login successful / failed).

```

saved_user = "admin"
saved_pass = "1234"
user = input("Enter username: ")
password = input("Enter password: ")
if user == saved_user and password == saved_pass:
    print("Login Successful")
else:
    print("Login Failed")

```

Logic Explanation

Program Logic:

- Predefined username and password are stored in variables saved_user and saved_pass.
- The user enters a username and password.
- The program checks two conditions using AND:
→ Entered username must match saved username.
→ Entered password must match saved password.
- If both conditions are true → Login Successful.

- Otherwise → Login Failed.

Mathematical Logic:

- Logical AND (and) returns true only when both conditions are true.
- Authentication works on exact matching of credentials.

33. Write a program to print index value and character present in given string

```
text = input("Enter a string: ")
index = 0
for ch in text:
    print("Index:", index, "Character:", ch)
    index += 1
```

Program Logic:

- A string is taken as input.
- A variable index starts from 0.
- A loop runs through each character of the string.
- In each loop:
 - Current index value is printed.
 - Corresponding character is printed.
- Index is increased by 1 after every iteration.

Mathematical Logic:

- String is a sequence of characters arranged by positions.
- Position counting starts from 0 and increases by 1.

34. write a program to display the alternat even number.

```
n = int(input("Enter limit: "))
count = 0
for i in range(2, n+1, 2):
    if count % 2 == 0:
        print(i, end=" ")
    count += 1
```

Logic Explanation

Program Logic:

- A limit is taken as input.
- Loop runs from 2 to limit with step 2 (only even numbers).
- A counter tracks how many even numbers have appeared.
- If counter is even (count % 2 == 0), the number is printed.
- Counter increases after each even number.

Mathematical Logic:

- Even numbers are multiples of 2.
- Alternate means selecting every second value.

35. write a program to display the alternat even number count the that is divisible by 4.

Program:

```
n = int(input("Enter limit: "))
count = 0
for i in range(2, n+1, 2):
    if i % 4 == 0:
        if count % 2 == 0:
            print(i, end=" ")
        count += 1
```

Logic Explanation

Program Logic:

- Loop generates even numbers.
- First check if number is divisible by 4.
- A counter tracks how many such numbers appear.
- Every second qualifying number is printed.

Mathematical Logic:

- Numbers divisible by 4 are multiples of 4.
- Alternate selection skips one value each time.

36. write a program to maximum digit form a given number.

Program

```
num = int(input("Enter a number: "))
max_digit = 0
while num > 0:
    digit = num % 10
    if digit > max_digit:
        max_digit = digit
    num //= 10
print("Maximum digit:", max_digit)
```

Logic Explanation

Program Logic:

- A number is taken as input.
- Loop extracts digits using % 10.
- Each digit is compared with max_digit.
- If digit is larger, it becomes new max.
- Number reduces using // 10.

Mathematical Logic:

- A number consists of digits (0–9).
- Maximum digit = largest among all digits.

37. write a program to swape the two number

Program

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
temp = a
a = b
b = temp
print("After swap:", a, b)
```

Logic Explanation

Program Logic:

- Two numbers are taken as input.
- A temporary variable stores the first number.
- First number is replaced with second.
- Second number is replaced with temp value.

Mathematical Logic: • Swapping exchanges positions of two values using temporary storage.

38. write a program to Check whether a number is a strong number or not.

Program

```
num = int(input("Enter a number: "))
temp = num
```



```

sum_fact = 0
while temp > 0:
    digit = temp % 10 # extract one-one digit from the given number
    #Calculate the factorial of each and every digit of given number
    fact = 1
    for i in range(1, digit+1):
        fact *= i
    sum_fact += fact
    temp //= 10
if sum_fact == num:
    print("Strong Number")
else:
    print("Not a Strong Number")

```

Logic Explanation

Program Logic:

- A number is taken as input and stored in num.
- A copy of the number is stored in temp for processing.
- A variable sum_fact is initialized to 0 to store the sum of factorials of digits.
- A loop runs while temp > 0.
- Each digit is extracted using temp % 10.
- The factorial of the digit is calculated using a loop from 1 to digit.
- This factorial is added to sum_fact.
- The digit is removed from the number using temp //= 10.
- After processing all digits, the sum of factorials is compared with the original number.
- If equal → Strong Number, otherwise not.

Mathematical Logic:

- A Strong Number is a number whose sum of factorial of digits = the number itself.
- Example: 145
 $\rightarrow 1! + 4! + 5! = 1 + 24 + 120 = 145$

Strong Numbers (in base 10)

1,2, 145, 40585 These are the **only four Strong Numbers**.

Number = 40585 $4! + 0! + 5! + 8! + 5! = 24 + 1 + 120 + 40320 + 120 = 40585 \rightarrow 0! = 1$

➔ write a program to Print all strong number within a given range (1 to 1000)

Solution:

```

print("Strong Numbers between 1 and 1000 are:")
for num in range(1, 1001):
    temp = num
    sum_fact = 0
    while temp > 0:
        digit = temp % 10
        # Find factorial of digit
        fact = 1
        for i in range(1, digit + 1):
            fact *= i
        sum_fact += fact
        temp //= 10
    if sum_fact == num:
        print(num, end=" ")

```

➔ Strong Numbers between 1 and 1000 are:

1 2 145

39. write a program to Find the square and cube of each number from 1 to 10.

Program:

```
for i in range(1, 11):
    square = i * i
    cube = i * i * i
    print(i, "Square:", square, "Cube:", cube)
```

Logic Explanation

Program Logic:

- A loop runs from 1 to 10.
- For each number i:
 - ➔ Square is calculated as $i \times i$.
 - ➔ Cube is calculated as $i \times i \times i$.
- Results are printed for each number.

Mathematical Logic:

- Square of number = n^2
- Cube of number = n^3

40. write a program to Print all perfect numbers within a given range.

Program

```
start = int(input("Enter start: "))
end = int(input("Enter end: "))
for num in range(start, end+1):
    sum_factors = 0
    for i in range(1, num):
        if num % i == 0:
            sum_factors += i
    if sum_factors == num:
        print(num, end=" ")
```

Logic Explanation

Program Logic:

- Start and end values of range are taken as input.
- Loop runs through each number in the range.
- For each number, sum of its factors (excluding the number itself) is calculated.
- Inner loop checks all numbers less than the number to see if they divide exactly.
- If sum of factors equals the number, it is printed as perfect number.

Mathematical Logic:

- Perfect Number = sum of its proper divisors equals the number.
- Example: 6 ➔ Factors: 1, 2, 3
 - ➔ Sum = $1 + 2 + 3 = 6$

Write a program to check whether a given number is perfect or not

```
num = int(input("Enter a number: "))
sum_factor = 0
for i in range(1, num):
    if num % i == 0:
        sum_factor += i
if sum_factor == num and num != 0:
    print(num, "is a Perfect Number")
else:
    print(num, "is Not a Perfect Number")
Enter a number: 6 ➔ 6 is a Perfect Number
```

Factors of 6: → 1, 2, 3

Sum: $1 + 2 + 3 = 6$ So, 6 is a Perfect Number.

28 Factors: → 1, 2, 4, 7, 14

Sum: $1 + 2 + 4 + 7 + 14 = 28$

Perfect Number → 6, 28, 496, 8128, 33550336

41. Write a Program to Check Whether a Given Number is a Palindrome or Not

Program:

```
num = int(input("Enter a number: "))
temp = num
rev = 0
while num > 0:
    digit = num % 10
    rev = rev * 10 + digit
    num //= 10
if temp == rev:
    print("Palindrome Number")
else:
    print("Not a Palindrome Number")
```

Sample Input: 121

Output: Palindrome Number

Logic Explanation: Palindrome Number

Program Logic:

- A number is taken as input and stored in num.
- The original number is saved in temp.
- The number is reversed using a loop.
- The reversed number is compared with the original number.
- If both are equal, it is a palindrome.

Mathematical Logic:

- A palindrome number is a number that remains the same when its digits are reversed.
- Reversing a number is done by extracting each digit from right to left.
- The last digit is obtained using modulo operation % 10.
- Each extracted digit is placed in reverse order by multiplying the current reversed value by 10 and adding the digit.
- If the reversed number is equal to the original number, the number is a palindrome.
- If both values are not equal, the number is not a palindrome.

Example:

- 121 → Reverse = 121 (Palindrome)
- 1331 → Reverse = 1331 (Palindrome)
- 545 → Reverse = 545 (Palindrome)

42. Write a Program to Print All Palindrome Numbers from 1 to 100

Program:

```
for num in range(1, 101):
    temp = num
```

```

rev = 0
while temp > 0:
    digit = temp % 10
    rev = rev * 10 + digit
    temp //= 10
if num == rev:
    print(num)

```

Output: 1 2 3 4 5 6 7 8 9 11 22 33 44 55 66 77 88 99

Logic Explanation: Palindrome Numbers from 1 to 100

Program Logic:

- Loop runs from 1 to 100.
- Each number is reversed.
- Original number is compared with reversed number.
- If equal, the number is printed.

Mathematical Logic:

- Palindrome numbers remain the same when reversed.

43. Write a Program to Check Whether a Given Number is an Armstrong Number or Not Armstrong Number (3-Digit Case)

Program:

```

num = int(input("Enter a number: "))
temp = num
sum = 0
while temp > 0:
    digit = temp % 10
    sum += digit ** 3
    temp //= 10
if sum == num:
    print("Armstrong Number")
else:
    print("Not an Armstrong Number")

```

Logic Explanation (3-Digit Armstrong Number)

Program Logic:

- A number is taken as input and stored in num.
- The original number is stored in temp.
- Each digit is extracted using % 10.
- The cube (digit³) of each digit is added to sum.
- After processing all digits, sum is compared with the original number.
- If both are equal, the number is an Armstrong number.

Mathematical Logic:

- A 3-digit Armstrong number satisfies: $abc = a^3 + b^3 + c^3$
- Example: $153 = 1^3 + 5^3 + 3^3 = 153$

Armstrong Number (General Case: 3-Digit, 4-Digit, 5-Digit, ...)

➔ **Program (Works for Any Number of Digits):**

```

num = int(input("Enter a number: "))
temp = num
sum = 0
digits = len(str(num))
while temp > 0:

```

```

digit = temp % 10
sum += digit ** digits
temp //= 10
if sum == num:
    print("Armstrong Number")
else:
    print("Not an Armstrong Number")

```

Logic Explanation (General Armstrong Number)

Program Logic:

- A number is taken as input and stored in num.
- The number of digits is calculated using len(str(num)).
- Each digit is extracted using % 10.
- Each digit is raised to the power of total digits.
- All powered digits are added to sum.
- The final sum is compared with the original number.

Mathematical Logic:

- An Armstrong number with **n digits** satisfies:

Number = sum of each digit raised to power n

- Examples:
- $153 = 1^3 + 5^3 + 3^3$
- $1634 = 1^4 + 6^4 + 3^4 + 4^4$
- This logic works for 3-digit, 4-digit, 5-digit, and higher numbers.

44. Write a Program to Find All Armstrong Numbers from 1 to 100

Program:

```

for num in range(1, 101):
    temp = num
    sum = 0
    while temp > 0:
        digit = temp % 10
        sum += digit ** 3
        temp //= 10
    if sum == num:
        print(num)

```

Output: 1 153

Logic Explanation: Armstrong Numbers from 1 to 100

Program Logic:

- Loop runs from 1 to 100.
- Each number is checked for Armstrong condition.
- If sum of cubes of digits equals the number, it is printed.

Mathematical Logic:

- Armstrong numbers satisfy the condition:

Sum of digit powers = number itself.

45. Write a Program to Check Whether a Given String is a Palindrome

Program:

```

s = input("Enter a string: ")
if s == s[::-1]:

```

Program-2:

```

s = input("Enter a string: ")
rev = ""
for ch in s:
    rev = ch + rev
if s == rev:
    print("Palindrome String")

```

```

        print("Palindrome String")
    else:
        print("Not a Palindrome String")

```

Logic Explanation: Palindrome String

Program Logic:

- A string is taken as input.
- The string is reversed character by character using a loop.
- Reversed string is stored in a new variable.
- The original string is compared with the reversed string.
- If both are equal, the string is a palindrome.

Mathematical / Logical Concept:

- A palindrome string reads the same from left to right and right to left.
- Reversing the order of characters does not change the string.
- Loop-based reversal checks each character position manually.
- If original and reversed strings are identical, the string is a palindrome.

Examples:

- madam → Palindrome
- level → Palindrome
- hello → Not a Palindrome

46. Write a program to find all palindrome string from given paragraph.

Program:

```

paragraph = input("Enter a paragraph: ").lower()
words = paragraph.split()           # Split paragraph into words convert in list
print("Palindrome words are:")
for i in words:                     # Loop through each word
    if i == i[::-1] and len(i) > 1: # Check palindrome
        print(i)

```

2nd Method

```

s = input("Enter a string: ").lower()
rev = ""
for ch in s:                         # Reverse string manually
    rev = ch + rev
if s == rev:
    print("Palindrome String")
else:
    print("Not a Palindrome String")

```

47. Write a Program to Find the Area of a Triangle

Program:

```

b = float(input("Enter base: "))
h = float(input("Enter height: "))
area = 0.5 * b * h
print("Area of triangle =", area)

```

Sample Input:

```

Base = 10
Height = 5

```

Output:

Area of triangle = 25.0

Logic Explanation: Area of Triangle

Program Logic:

- Base and height are taken as input.
- Formula is applied to calculate area.
- The result is printed.

Mathematical Logic:

- Area of triangle = $\frac{1}{2} \times \text{base} \times \text{height}$.

48. Write a Program to Find the Area of a Square**Program:**

```
side = float(input("Enter side: "))
area = side * side
print("Area of square =", area)
```

Sample Input: 4**Output:** Area of square = 16.0**Logic Explanation: Area of Square****Program Logic:**

- Side of square is taken as input.
- Side is multiplied by itself.
- Area is printed.

Mathematical Logic: • Area of square = side $\times$ side.**49. Write a Program to Print Numbers Whose Digits Are in Increasing Order****Program:**

```
num = input("Enter a number: ")      # Take number as string
for i in range(len(num) - 1):        # Loop through digits
    if num[i] >= num[i + 1]:          # Compare current and next digit
        print("Digits are not in increasing order")
        break                        # Stop if condition fails
    else:
        print("Digits are in increasing order") # Runs if loop doesn't break
```

Sample Input: 12349**Output:** Digits are in increasing order**Logic Explanation: Digits in Increasing Order****Program Logic:**

- Number is taken as string input.
- Each digit is compared with the next digit.
- If any digit is greater than or equal to the next, order fails.
- If all digits follow increasing order, result is printed.

Mathematical Logic:

- Each digit must be smaller than the digit following it.
- Example: 1234 ✓, 1223 ✗, 321 ✗.

50. Write a Program to Find the Smallest Digit in a Number**Program:**

```
num = int(input("Enter a number: "))
small = num % 10    # Take last digit as initial smallest
num //= 10
while num > 0:
    digit = num % 10
```

Program Logic:

- A number is taken as input.
- Digits are extracted one by one using % 10.
- Each digit is compared with the current smallest value.
- The smallest digit is stored and printed.

Mathematical Logic:

- Comparing all digits gives the minimum value.

```

    if digit < small:
        small = digit
    num //= 10
print("Smallest digit =", small)

```

Sample Input: 591342

Output: Smallest digit = 1

Logic Explanation: Smallest Digit in a Number

51. Write a Program to Check Whether a Given Number is Prime or Not

Program:

```

num = int(input("Enter a number: "))
if num <= 1:
    print("Not a Prime Number")
else:
    for i in range(2, num):
        if num % i == 0:
            print("Not a Prime Number")
            break
    else:
        print("Prime Number")

```

Sample Input: 7

Output: Prime Number

Logic Explanation: Prime Number (Using for-else)

Program Logic:

- A number is taken as input.
- Numbers less than or equal to 1 are not prime.
- A loop checks divisibility from 2 to (n-1).
- If a divisor is found, break stops the loop and prints *Not Prime*.
- If the loop completes without break, the else block executes and prints *Prime Number*.

Mathematical Logic:

- A prime number has exactly two factors: 1 and itself.
- If no other divisor exists, the number is prime.
- The for-else structure confirms absence of divisors.

Program-2: with flag variable

```

num = int(input("Enter a number: "))
flag = True
if num <= 1:
    flag = False
else:
    for i in range(2, num):
        if num % i == 0:
            flag = False
            break
if flag:
    print("Prime Number")
else:
    print("Not a Prime Number")

```

Sample Input: 7

Output: Prime Number

Logic Explanation: Prime Number Check

Program Logic:

- A number is taken as input.
- Numbers less than or equal to 1 are not prime.
- Loop checks divisibility from 2 to (n-1).
- If any divisor is found, the number is not prime.

Mathematical Logic:

- A prime number has exactly two factors: 1 and itself.
- If divisible by any other number, it is not prime.

52. Write a Program to Print All Prime Numbers in a Given Interval

Program:

```
start = int(input("Enter start value: "))
end = int(input("Enter end value: "))
for num in range(start, end + 1):
    if num > 1:
        for i in range(2, num):
            if num % i == 0:
                break
        else:
            print(num)
```

Sample Input:

Start = 1

End = 50

Output: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47

Logic Explanation: Prime Numbers in Interval

Program Logic:

- Loop runs through the given interval.
- Each number is checked for prime condition.
- If no divisor is found, the number is printed.

Mathematical Logic:

- Prime numbers are divisible only by 1 and themselves.

53. Write a Program to Find Count and Sum of All Prime Numbers from 50 to 500

Program:

```
count = 0
sum = 0
for num in range(50, 501):
    if num > 1:
        for i in range(2, num):
            if num % i == 0:
                break
        else:
            count += 1
            sum += num
print("Count =", count)
print("Sum =", sum)
```

Output:

Count = 95

Sum = 21536

Logic Explanation: Count and Sum of Prime Numbers

Program Logic:

- Loop runs from 50 to 500.
- Each number is checked for primality.
- Prime numbers are counted and added.

Mathematical Logic:

- Only numbers with two distinct factors are considered prime.

54. Write a Program to Find the Nth Prime Number

Program:

```

n = int(input("Enter value of n: "))
count = 0
num = 2
while True:
    for i in range(2, num):
        if num % i == 0:
            break
    else:
        count += 1
        if count == n:
            print("Nth prime number =", num)
            break
    num += 1

```

Sample Input: n = 5

Output: Nth prime number = 11

Logic Explanation: Nth Prime Number

Program Logic:

- Input n specifies which prime number is required.
- Numbers are checked one by one starting from 2.
- Each prime increases the counter.
- When count equals n, the prime is printed.

Mathematical Logic:

- Prime numbers occur at specific positions in natural numbers.
- The nth prime is found by counting primes sequentially.

55. Write a Program to Find the Prime Factors of a Given Number

Program:

```

num = int(input("Enter a number: "))
for i in range(2, num + 1):
    if num % i == 0:
        for j in range(2, i):
            if i % j == 0:
                break
        else:
            print(i)

```

Sample Input: 60

Output: 2 3 5

Logic Explanation: Prime Factors

Program Logic:

- Input number is taken.
- Loop checks all possible factors.
- Each factor is tested for primality.
- Prime factors are printed.

Mathematical Logic:

- Prime factors are prime numbers that divide the given number exactly.
- Example: $60 = 2 \times 2 \times 3 \times 5$.

56. Write a Program to Check Whether the Given Numbers are Co-Prime or Not

Program:

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
gcd = 1
for i in range(1, min(a, b) + 1):
    if a % i == 0 and b % i == 0:
        gcd = i
```

```
if gcd == 1:
    print("Co-Prime Numbers")
else:
    print("Not Co-Prime Numbers")
```

Sample Input: 8 15

Output: Co-Prime Numbers

Logic Explanation: Co-Prime Numbers

Program Logic:

- Two numbers are taken as input.
- Common divisors are checked using a loop.
- Greatest common divisor (GCD) is calculated.
- If GCD is 1, numbers are co-prime.

Mathematical Logic:

- Co-prime numbers have only one common factor, which is 1.
- Example: (8, 15), (9, 10).

57. Write a Program to Display the Nearest Prime Number for a Given Number

Program:

```
num = int(input("Enter a number: "))
def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
lower = num - 1
upper = num + 1
while True:
    if is_prime(lower):
        print("Nearest prime number =", lower)
```

```

        break
    elif is_prime(upper):
        print("Nearest prime number =", upper)
        break
    lower -= 1
    upper += 1

```

Sample Input:20

Output: Nearest prime number = 19

Logic Explanation: Nearest Prime Number

Program Logic:

- Start checking numbers just below and above the given number.
- Check each number for prime condition.
- The first prime encountered is printed.

Mathematical Logic:

- Prime numbers are integers divisible only by 1 and themselves.
- The nearest prime has the smallest difference from the given number.

58. Write a Program to Check Whether the Nearest Prime Number is Palindrome or Not

Program:

```

num = int(input("Enter a number: "))
def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
n = num
while True:
    n += 1
    if is_prime(n):
        prime = n
        break
temp = prime
rev = 0
while temp > 0:
    digit = temp % 10
    rev = rev * 10 + digit
    temp //= 10
if prime == rev:
    print("Nearest prime is Palindrome")
else:
    print("Nearest prime is Not Palindrome")

```

Sample Input: 30

Output: Nearest prime is Palindrome

Logic Explanation: Palindrome Prime

Program Logic:

- Nearest prime number is found.

- The prime number is reversed.
- Original and reversed values are compared.

Mathematical Logic:

- A palindrome reads the same forward and backward.
- Example: 101, 131 are palindrome primes.

59. Write a Program to Find the GCD and LCM of Two Given Numbers

Program:

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
gcd = 1
for i in range(1, min(a, b) + 1):
    if a % i == 0 and b % i == 0:
        gcd = i
lcm = (a * b) // gcd
print("GCD =", gcd)
print("LCM =", lcm)
```

Sample Input: 12 18

Output:

GCD = 6
LCM = 36

Logic Explanation: GCD and LCM

Program Logic:

- Common divisors are checked to find GCD.
- LCM is calculated using the formula.

Mathematical Logic:

- GCD = greatest common divisor.
- $LCM \times GCD = \text{product of two numbers.}$

60. Write a Program to Find the Largest Prime Number Within a Given Range

Program:

```
start = int(input("Enter start: "))
end = int(input("Enter end: "))
largest = None
for num in range(start, end + 1):
    if num > 1:
        for i in range(2, num):
            if num % i == 0:
                break
        else:
            largest = num
if largest:
    print("Largest prime number =", largest)
else:
    print("No prime number in range")
```

Sample Input: 10 50

Output: Largest prime number = 47

Logic Explanation: Largest Prime in Range

Program Logic:

- Loop runs through the given range.
- Each number is checked for prime condition.
- The latest prime found is stored as largest.

Mathematical Logic:

- Prime numbers have exactly two factors.
- The highest prime within the interval is the largest prime.

61. Write a Program to Check Whether a Number is a Power of 2 or Not**Program**

```
num = int(input("Enter a number: "))
```

```
if num > 0 and (num & (num - 1)) == 0:
```

```
    print("Power of 2")
```

```
else:
```

```
    print("Not a Power of 2")
```

Sample Input

16

Output

Power of 2

Logic Explanation: Power of 2**Program Logic**

- A number is taken as input and stored in variable num.
- First, the program checks that the number is greater than 0.
- The expression (num & (num - 1)) is evaluated.
- If the result is 0, the number is a power of 2.
- Otherwise, it is not a power of 2.

Mathematical Logic

- A power of 2 can be written as 2^n , where $n \geq 0$.
- Binary representation of a power of 2 contains only one 1.
- Subtracting 1 flips all bits after the first 1.
- AND operation between the number and (number - 1) becomes 0.
- Example:

16 → 10000

15 → 01111

AND → 00000

62. Write a Program to Count the Number of Digits in a Given Number**Program**

```
num = int(input("Enter a number: "))
```

```
count = 0
```

```
while num > 0:
```

```
    count += 1
```

```
    num //= 10
```

```
print("Number of digits:", count)
```

Sample Input

45896

Output

Number of digits: 5

Logic Explanation: Count Digits**Program Logic**

- A number is taken as input.
- A counter variable is initialized to zero.
- In each iteration, the last digit is removed using integer division by 10.
- The counter increases by 1 for each digit removed.
- Loop continues until the number becomes zero.

Mathematical Logic

- Dividing a number by 10 removes its last digit.
- Each division represents one digit.
- Total number of divisions equals the total number of digits.

63. Write a Program to Find the Sum of First N Natural Numbers**Program**

```
n = int(input("Enter a number: "))
sum = 0
```

```
for i in range(1, n + 1):
    sum += i
```

```
print("Sum:", sum)
```

Sample Input 10

Output Sum: 55

Logic Explanation: Sum of Natural Numbers**Program Logic**

- A number n is taken as input.
- A loop runs from 1 to n.
- Each number is added to the sum variable.
- After the loop ends, the total sum is displayed.

Mathematical Logic

- Natural numbers start from 1.
- The sum of first n natural numbers is:

$$n(n + 1) / 2$$

- Example:

$$10 \times 11 \div 2 = 55$$

64. Write a Program to Find the Sum of Digits Until It Becomes a Single Digit**Program**

```
num = int(input("Enter a number: "))
while num >= 10:
    s = 0
    while num > 0:
        s += num % 10
        num //= 10
```

```
num = s
print("Single digit sum:", num)
```

Sample Input:- 987

Output:- Single digit sum: 6

Logic Explanation: Single Digit Sum

Program Logic

- A number is taken as input.
- Digits are extracted using % 10 and added.
- After summing digits, the number is updated.
- This process repeats until a single digit remains.

Mathematical Logic

- Sum of digits reduces the number size.
- Repeating digit sum gives a **digital root**.
- Example:
 $9+8+7 = 24$
 $2+4 = 6$

65. Write a Program to Check Whether a Number is a Harshad (Niven) Number or Not

Program

```
num = int(input("Enter a number: "))
temp = num
sum_digits = 0
while temp > 0:
    sum_digits += temp % 10
    temp //= 10
if num % sum_digits == 0:
    print("Harshad Number")
else:
    print("Not a Harshad Number")
```

Sample Input: 18

Output: Harshad Number

Logic Explanation: Harshad Number

Program Logic

- A number is taken as input.
- Sum of its digits is calculated.
- The original number is divided by the digit sum.
- If divisible, it is a Harshad number.

Mathematical Logic

- A Harshad number is divisible by the sum of its digits.
- Example:
 $18 \rightarrow 1+8 = 9$
 $18 \div 9 = 2$

66. Write a Program to Print All Harshad Numbers from 1 to 100

Program

```
for num in range(1, 101):
    temp = num
    s = 0
    while temp > 0:
```

Example 1

$24 \rightarrow 2 + 4 = 6$

$24 \div 6 = 4$

Example 2

$36 \rightarrow 3 + 6 = 9$

$36 \div 9 = 4$

Example 3

$42 \rightarrow 4 + 2 = 6$

$42 \div 6 = 7$

Example 4

$63 \rightarrow 6 + 3 = 9$

$63 \div 9 = 7$

Example 5

$80 \rightarrow 8 + 0 = 8$

$80 \div 8 = 10$

Example 6

$111 \rightarrow 1 + 1 + 1 = 3$

$111 \div 3 = 37$


```

s += temp % 10
temp //= 10
if num % s == 0:
    print(num, end=" ")

```

Output

1 2 3 4 5 6 7 8 9 10 12 18 20 21 24 27 30 36 40 42 45 48 50 54 60 63 70 72 80 81 84 90 100

Logic Explanation: Harshad Numbers

Program Logic

- A loop runs from 1 to 100.
- For each number, the sum of its digits is calculated.
- The number is checked for divisibility by its digit sum.
- If divisible, the number is printed.

Mathematical Logic

- A Harshad number is divisible by the sum of its digits.
- Example:

$18 \rightarrow 1+8 = 9$

$18 \div 9 = 2$

67. Write a Program to Find the Largest Digit in a Given Number

Program

```

num = int(input("Enter a number: "))
largest = 0
while num > 0:
    digit = num % 10
    if digit > largest:
        largest = digit
    num //= 10
print("Largest digit:", largest)

```

Sample Input 45826

Output Largest digit: 8

Logic Explanation: Largest Digit

Program Logic

- A number is taken as input.
- Digits are extracted one by one using % 10.
- Each digit is compared with the current largest digit.
- The largest value is updated accordingly.

Mathematical Logic

- Comparing digits determines the maximum value.
- The digit with the highest value (0–9) is the largest.

68. Write a Program to Check Whether Two Numbers are Equal Without Using Comparison Operators

Program

```

a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
if a ^ b == 0:
    print("Numbers are Equal")
else:
    print("Numbers are Not Equal")

```

Sample Input 5 5

Output Numbers are Equal

Logic Explanation: Equality Without Comparison

Program Logic

- Two numbers are taken as input.
- XOR (^) operation is applied.
- If result is 0, numbers are equal.
- Otherwise, numbers are different.

Mathematical Logic

- XOR of two identical numbers is 0.
- XOR of different numbers is non-zero.
- This logic avoids comparison operators.

69. Write a Program to Convert a Decimal Number to Binary

Program

```
num = int(input("Enter a decimal number: "))
binary = ""
while num > 0:
    binary = str(num % 2) + binary
    num //= 2
print("Binary:", binary)
```

Sample Input 10

Output Binary: 1010

Logic Explanation: Decimal to Binary

Program Logic

- A decimal number is taken as input.
- The number is repeatedly divided by 2.
- Remainders are collected in reverse order.
- Final binary number is formed.

Mathematical Logic

- Binary system uses base 2.
- Division by 2 gives binary digits (0 or 1).
- Remainders read from last to first form the binary number.

70. Write a Program to Convert a Binary Number to Decimal

Program

```
binary = input("Enter binary number: ")
decimal = 0
power = 0
for digit in binary[::-1]:
    decimal += int(digit) * (2 ** power)
    power += 1
print("Decimal:", decimal)
```

Sample Input 1010

Output Decimal: 10

Logic Explanation: Binary to Decimal

Program Logic

- A binary number is taken as input.
- Digits are processed from right to left.
- Each digit is multiplied by 2 raised to its position.
- The results are added to get the decimal value.

Mathematical Logic

- Binary digits represent powers of 2.

- Example:

$$1010 = (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (0 \times 2^0)$$

$$= 8 + 0 + 2 + 0 = 10$$

81. Write a Program to Print the First N Prime Numbers

Program

```
n = int(input("Enter value of N: "))
count = 0
num = 2
while count < n:
    for i in range(2, num):
        if num % i == 0:
            break
    else:
        print(num, end=" ")
        count += 1
        num += 1
```

Sample Input 5

Output 2 3 5 7 11

Logic Explanation: First N Prime Numbers

Program Logic

- A number n is taken as input.
- Starting from 2, each number is checked for primality.
- If a number has no divisors, it is printed.
- The process continues until n prime numbers are printed.

Mathematical Logic

- Prime numbers have exactly two factors: 1 and itself.
- Counting stops after n primes are found.

82. Write a Program to Print Numbers Divisible by Both 2 and 3 from 1 to 100

Program

```
for i in range(1, 101):
    if i % 2 == 0 and i % 3 == 0:
        print(i, end=" ")
```

Output 6 12 18 24 30 36 42 48 54 60 66 72 78 84 90 96

Logic Explanation: Divisible by 2 and 3

Program Logic

- Loop runs from 1 to 100.
- Each number is checked for divisibility by 2 and 3.
- If both conditions are true, the number is printed.

Mathematical Logic

- A number divisible by both 2 and 3 is divisible by **LCM(2,3) = 6**.

83. Write a Program to Find the Sum of Odd Digits in a Number

Program

```
num = int(input("Enter a number: "))
sum_odd = 0
```

```

while num > 0:
    digit = num % 10
    if digit % 2 != 0:
        sum_odd += digit
    num //= 10
print("Sum of odd digits:", sum_odd)

```

Sample Input 45893

Output Sum of odd digits: 17

Logic Explanation: Sum of Odd Digits

Program Logic

- A number is taken as input.
- Digits are extracted one by one.
- If a digit is odd, it is added to the sum.
- Final sum is displayed.

Mathematical Logic

- Odd digits are not divisible by 2.
- Only digits with remainder 1 when divided by 2 are added.

84. Write a Program to Check Whether a Number is a Spy Number or Not

Program

```

num = int(input("Enter a number: "))
temp = num
sum_digits = 0
product = 1
while temp > 0:
    digit = temp % 10
    sum_digits += digit
    product *= digit
    temp //= 10
if sum_digits == product:
    print("Spy Number")
else:
    print("Not a Spy Number")

```

Sample Input 1124

Output Spy Number

Logic Explanation: Spy Number

Program Logic

- A number is taken as input.
- Sum and product of its digits are calculated.
- If sum equals product, it is a spy number.

Mathematical Logic

- Spy number condition:

Sum of digits = Product of digits

- Example:

1124 → Sum = 1+1+2+4 = 8

Product = 1×1×2×4 = 8

85. Write a Program to Print All Spy Numbers Between 1 and 100

Program

```

for num in range(1, 101):

```

```

temp = num
s = 0
p = 1
while temp > 0:
    digit = temp % 10
    s += digit
    p *= digit
    temp //= 10
if s == p:
    print(num, end=" ")

```

Output 1 2 3 4 5 6 7 8 9 22

Logic Explanation: Spy Numbers

Program Logic

- Loop runs from 1 to 100.
- For each number, sum and product of digits are calculated.
- Numbers satisfying spy condition are printed.

Mathematical Logic

- A spy number satisfies:

Sum of digits = Product of digits

86. Write a Program to Count How Many Even and Odd Digits Are Present in a Number

Program

```

num = int(input("Enter a number: "))
even_count = 0
odd_count = 0
while num > 0:
    digit = num % 10
    if digit % 2 == 0:
        even_count += 1
    else:
        odd_count += 1
    num //= 10
print("Even digits:", even_count)
print("Odd digits:", odd_count)

```

Sample Input 45826

Output

```

Even digits: 3
Odd digits: 2

```

Logic Explanation: Even and Odd Digits

Program Logic

- A number is taken as input.
- Digits are extracted one by one.
- Each digit is checked for divisibility by 2.
- Even and odd digit counters are increased accordingly.

Mathematical Logic

- Even digits are divisible by 2.
- Odd digits leave remainder 1 when divided by 2.

87. Write a Program to Check Whether a Number is a Duck Number or Not

Program

```

num = input("Enter a number: ")
if '0' in num[1:]:
    print("Duck Number")
else:
    print("Not a Duck Number")

```

Sample Input 1023

Output Duck Number

Logic Explanation: Duck Number

Program Logic

- Number is taken as a string.
- Leading zero is ignored.
- Presence of digit 0 is checked.
- If found, number is a Duck number.

Mathematical Logic

- A Duck number contains at least one zero.
- Leading zero is not considered.
- Example: 1023 ✓, 0123 ✗

88. Write a Program to Print All Duck Numbers from 1 to 100

Program

```

for num in range(1, 101):
    s = str(num)
    if '0' in s[1:]:
        print(num, end=" ")

```

Output 10 20 30 40 50 60 70 80 90 100

Logic Explanation: Duck Numbers from 1 to 100

Program Logic

- Loop runs from 1 to 100.
- Each number is converted to string.
- Leading digit is ignored.
- Numbers containing zero are printed.

Mathematical Logic

- Duck numbers must contain digit 0 except as the first digit.

89. Write a Program to Find the Difference Between the Sum of Even and Odd Digits

Program

```

num = int(input("Enter a number: "))
sum_even = 0
sum_odd = 0
while num > 0:
    digit = num % 10
    if digit % 2 == 0:
        sum_even += digit
    else:
        sum_odd += digit
    num //= 10
print("Difference:", abs(sum_even - sum_odd))

```

Sample Input 45826

Output Difference: 7

Logic Explanation: Difference of Digit Sums

Program Logic

- A number is taken as input.
- Digits are separated using % 10.
- Even and odd digit sums are calculated.
- Absolute difference is displayed.

Mathematical Logic

- Difference = |Sum of even digits – Sum of odd digits|

90. Write a Program to Print Numbers Whose Sum of Digits is Even

Program

```
for num in range(1, 101):
```

```
    temp = num
```

```
    s = 0
```

```
    while temp > 0:
```

```
        s += temp % 10
```

```
        temp //= 10
```

```
    if s % 2 == 0:
```

```
        print(num, end=" ")
```

Output 2 4 6 8 11 13 15 17 19 20 ... 100

Logic Explanation: Sum of Digits Even

Program Logic

- Loop runs from 1 to 100.
- Sum of digits is calculated for each number.
- If sum is even, the number is printed.

Mathematical Logic

- Even numbers are divisible by 2.
- Sum of digits determines parity.

91. Write a Program to Count Uppercase and Lowercase Letters in a String

Program

```
s = input("Enter a string: ")
```

```
upper = 0
```

```
lower = 0
```

```
for ch in s:
```

```
    if ch.isupper():
```

```
        upper += 1
```

```
    elif ch.islower():
```

```
        lower += 1
```

```
print("Uppercase letters:", upper)
```

```
print("Lowercase letters:", lower)
```

Sample Input: HeLLo World

Output

Uppercase letters: 3

Lowercase letters: 7

Logic Explanation: Uppercase and Lowercase Letters

Program Logic

- A string is taken as input.
- Each character is checked using `isupper()` and `islower()`.
- Uppercase and lowercase counters are increased accordingly.

Mathematical / Logical Concept

- English alphabets have two cases: uppercase and lowercase.
- Built-in functions identify character case accurately.

92. Write a Program to Remove Vowels from a Given String

Program

```
s = input("Enter a string: ")
result = ""
for ch in s:
    if ch.lower() not in "aeiou":
        result += ch
print("String without vowels:", result)
```

Sample Input: Programming

Output: String without vowels: Prgrmmng

Logic Explanation: Remove Vowels

Program Logic

- A string is taken as input.
- Each character is checked against vowel set.
- Non-vowel characters are added to result string.

Mathematical / Logical Concept

- Vowels are a, e, i, o, u (both cases).
- Filtering removes unwanted characters.

93. Write a Program to Check Whether Two Strings are Anagrams or Not

Program

```
s1 = input("Enter first string: ")
s2 = input("Enter second string: ")
if sorted(s1) == sorted(s2):
    print("Anagram Strings")
else:
    print("Not Anagram Strings")
```

Sample Input

listen
silent

Output: Anagram Strings

Logic Explanation: Anagram Strings

Program Logic

- Two strings are taken as input.
- Characters of both strings are sorted.
- Sorted strings are compared for equality.

Mathematical / Logical Concept

- Anagrams contain the same characters with same frequency.
- Order does not matter, only character count matters.

94. Write a Program to Find the Frequency of Each Character in a String

Program

```
s = input("Enter a string: ")
for ch in set(s):
    print(ch, ":", s.count(ch))
```

Sample Input apple

Output

```
a : 1
p : 2
l : 1
e : 1
```

Logic Explanation: Character Frequency

Program Logic

- A string is taken as input.
- Unique characters are obtained using set().
- Frequency of each character is counted using count().

Mathematical / Logical Concept

- Frequency means number of occurrences.
- Counting helps analyze character distribution.

95. Write a Program to Replace All Spaces in a String with Hyphens (-)

Program

```
s = input("Enter a string: ")
result = s.replace(" ", "-")
print("Modified string:", result)
```

Sample Input : Python Programming Language

Output: Modified string: Python-Programming-Language

Logic Explanation: Replace Spaces

Program Logic

- A string is taken as input.
- replace() function replaces spaces with hyphens.
- Modified string is printed.

Mathematical / Logical Concept

- String replacement substitutes one character sequence with another.

96. Write a Program to Reverse Each Word in a Given String

Program

```
s = input("Enter a string: ")
words = s.split()
result = []
for w in words:
    result.append(w[::-1])
print("Reversed words:", " ".join(result))
```

Sample Input: Python Programming

Output: Reversed words: nohtyP gnimmargorP

Logic Explanation: Reverse Each Word

Program Logic

- A string is taken as input.
- The string is split into words using `split()`.
- Each word is reversed using slicing `[::-1]`.
- Reversed words are joined back into a string.

Mathematical / Logical Concept

- Reversing rearranges characters in opposite order.
- Word boundaries remain unchanged.

97. Write a Program to Find the Longest Word in a String

Program

```
s = input("Enter a string: ")
words = s.split()
longest = words[0]
```

```
for w in words:
    if len(w) > len(longest):
        longest = w
print("Longest word:", longest)
```

Sample Input:- Python is very powerful

Output:- Longest word: powerful

Logic Explanation: Longest Word

Program Logic

- A string is taken as input.
- It is split into words.
- Length of each word is compared.
- Word with maximum length is stored and printed.

Mathematical / Logical Concept

- Length comparison determines maximum size.
- The word with highest character count is the longest.

98. Write a Program to Check Whether a String Contains Only Digits

Program

```
s = input("Enter a string: ")
if s.isdigit():
    print("String contains only digits")
else:
    print("String does not contain only digits")
```

Sample Input 12345

Output: String contains only digits

Logic Explanation: Only Digits Check

Program Logic

- A string is taken as input.
- `isdigit()` checks whether all characters are digits.
- Result is printed based on condition.

Mathematical / Logical Concept

- Digits range from 0 to 9.
- Presence of any non-digit fails the condition.

99. Write a Program to Count the Number of Words in a Sentence

Program

```
s = input("Enter a sentence: ")
words = s.split()
print("Number of words:", len(words))
```

Sample Input

Python is easy to learn

Output

Number of words: 5

Logic Explanation: Word Count

Program Logic

- A sentence is taken as input.
- split() separates words based on spaces.
- Total number of words is counted using len().

Mathematical / Logical Concept

- Each space-separated unit is considered a word.
- Counting elements gives word count.

100. Write a Program to Capitalize the First Letter of Each Word in a String

Program

```
s = input("Enter a string: ")
result = s.title()
print("Capitalized string:", result)
```

Sample Input

python programming language

Output

Capitalized string: Python Programming Language

Logic Explanation: Capitalize Words

Program Logic

- A string is taken as input.
- title() converts the first letter of each word to uppercase.
- Modified string is printed.

Mathematical / Logical Concept

- Capitalization changes character case.
- First letter of every word is converted to uppercase.

96. Write a Program to Reverse Each Word in a Given String

Program

```
s = input("Enter a string: ")
words = s.split()
result = []
for w in words:
    result.append(w[::-1])
print("Reversed words:", " ".join(result))
```

Sample Input: Python Programming

Output: Reversed words: nohtyP gnimmargorP

Logic Explanation: Reverse Each Word

Program Logic

- A string is taken as input.
- The string is split into words using `split()`.
- Each word is reversed using slicing `[::-1]`.
- Reversed words are joined back into a string.

Mathematical / Logical Concept

- Reversing rearranges characters in opposite order.
- Word boundaries remain unchanged.

97. Write a Program to Find the Longest Word in a String

Program

```
s = input("Enter a string: ")
words = s.split()
longest = words[0]
for w in words:
    if len(w) > len(longest):
        longest = w
```

```
print("Longest word:", longest)
```

Sample Input: Python is very powerful

Output: Longest word: powerful

Logic Explanation: Longest Word

Program Logic

- A string is taken as input.
- It is split into words.
- Length of each word is compared.
- Word with maximum length is stored and printed.

Mathematical / Logical Concept

- Length comparison determines maximum size.
- The word with highest character count is the longest.

98. Write a Program to Check Whether a String Contains Only Digits

Program

```
s = input("Enter a string: ")
if s.isdigit():
    print("String contains only digits")
else:
    print("String does not contain only digits")
```

Sample Input: 12345

Output: String contains only digits

Logic Explanation: Only Digits Check

Program Logic

- A string is taken as input.
- `isdigit()` checks whether all characters are digits.
- Result is printed based on condition.

Mathematical / Logical Concept

- Digits range from 0 to 9.
- Presence of any non-digit fails the condition.

99. Write a Program to Count the Number of Words in a Sentence

Program

```
s = input("Enter a sentence: ")
words = s.split()
print("Number of words:", len(words))
```

Sample Input: Python is easy to learn

Output: Number of words: 5

Logic Explanation: Word Count

Program Logic

- A sentence is taken as input.
- split() separates words based on spaces.
- Total number of words is counted using len().

Mathematical / Logical Concept

- Each space-separated unit is considered a word.
- Counting elements gives word count.

100. Write a Program to Capitalize the First Letter of Each Word in a String

Program

```
s = input("Enter a string: ")
result = s.title()
print("Capitalized string:", result)
```

Sample Input: python programming language

Output:- Capitalized string: Python Programming Language

Logic Explanation: Capitalize Words

Program Logic

- A string is taken as input.
- title() converts the first letter of each word to uppercase.
- Modified string is printed.

Mathematical / Logical Concept

- Capitalization changes character case.
- First letter of every word is converted to uppercase.

101. Write a Program to Check Whether a Number is an Automorphic Number or Not

Program

```
num = int(input("Enter a number: "))
square = num * num
if str(square).endswith(str(num)):
    print("Automorphic Number")
else:
    print("Not an Automorphic Number")
```

Sample Input: 25

Output: Automorphic Number

Logic Explanation: Automorphic Number

Program Logic

- The number is taken as input.
- Its square is calculated.
- The last digits of the square are compared with the original number.
- If both match, the number is automorphic.

Mathematical Logic

- An **Automorphic number** is a number whose **square ends with the same digits as the number itself**.

- If a number has k digits, then the last k digits of its square must be equal to the number.
- Example:
 $25^2 = 625 \rightarrow$ last two digits = 25
- Hence, 25 satisfies the automorphic condition.

102. Write a Program to Print All Automorphic Numbers from 1 to 1000

Program

```
for num in range(1, 1001):
    square = num * num
    if str(square).endswith(str(num)):
        print(num, end=" ")
```

Output: 1 5 6 25 76 376 625

Logic Explanation: Automorphic Numbers from 1 to 1000

Program Logic

- A loop runs from 1 to 1000.
- Square of each number is calculated.
- If the square ends with the number, it is printed.

Mathematical Logic

- Only numbers satisfying the **automorphic property** are selected.
- The condition depends on digit-matching between a number and its square.
- Automorphic numbers are rare and pattern-based.

103. Write a Program to Find the Sum of Prime Digits in a Number

Program

```
num = int(input("Enter a number: "))
sum_prime = 0
while num > 0:
    digit = num % 10
    if digit > 1:
        for i in range(2, digit):
            if digit % i == 0:
                break
        else:
            sum_prime += digit
    num //= 10
print("Sum of prime digits:", sum_prime)
```

Sample Input: 23954

Output: Sum of prime digits: 19

Logic Explanation: Sum of Prime Digits

Program Logic

- Digits of the number are extracted one by one.
- Each digit is checked for primality.
- Prime digits are added to the sum.

Mathematical Logic

- A **prime number** has exactly two factors: 1 and itself.
- Prime digits are: **2, 3, 5, 7**.
- Example:

23954 → Prime digits = 2, 3, 5

Sum = 2 + 3 + 5 = 10

104. Write a Program to Check Whether a Number is a Disarium Number or Not

Program

```
num = int(input("Enter a number: "))
digits = list(map(int, str(num)))
sum_power = 0
for i in range(len(digits)):
    sum_power += digits[i] ** (i + 1)
```

```
if sum_power == num:
    print("Disarium Number")
else:
    print("Not a Disarium Number")
```

Sample Input 135

Output: Disarium Number

Logic Explanation: Disarium Number

Program Logic

- Digits are extracted in order.
- Each digit is raised to the power of its position.
- The powered values are added.
- If the sum equals the number, it is Disarium.

Mathematical Logic

- A **Disarium number** satisfies:

Sum of (digit raised to its positional power) = number

- Example:

135 → $1^1 + 3^2 + 5^3$

= 1 + 9 + 125

= 135

105. Write a Program to Print All Disarium Numbers from 1 to 100

Program

```
for num in range(1, 101):
    digits = list(map(int, str(num)))
    sum_power = 0
    for i in range(len(digits)):
        sum_power += digits[i] ** (i + 1)
    if sum_power == num:
        print(num, end=" ")
```

Output:- 1 2 3 4 5 6 7 8 9 89

Logic Explanation: Disarium Numbers from 1 to 100

Program Logic

- Loop runs from 1 to 100.
- Positional power sum is calculated for each number.
- Valid Disarium numbers are printed.

Mathematical Logic

- Single-digit numbers are always Disarium ($\text{digit}^1 = \text{digit}$).
- 89 is Disarium because:
 $8^1 + 9^2 = 8 + 81 = 89 \checkmark$

106. Write a Program to Check Whether a Number is a Peterson Number or Not

Program

```
num = int(input("Enter a number: "))
temp = num
sum_fact = 0
while temp > 0:
    digit = temp % 10
    fact = 1
    for i in range(1, digit + 1):
        fact *= i
    sum_fact += fact
    temp //= 10
if sum_fact == num:
    print("Peterson Number")
else:
    print("Not a Peterson Number")
```

Sample Input:- 145

Output: Peterson Number

Logic Explanation: Peterson Number

Program Logic

- A number is taken as input and stored in num.
- Each digit of the number is extracted one by one.
- Factorial of each digit is calculated.
- All digit factorials are added together.
- If the sum equals the original number, it is a Peterson number.

Mathematical Logic

- A Peterson number is a number equal to the **sum of factorials of its digits**.
- Factorial of a digit d is:

$$d! = d \times (d-1) \times \dots \times 1$$

- Example:

$$145 \rightarrow 1! + 4! + 5!$$

$$= 1 + 24 + 120$$

$$= 145$$

107. Write a Program to Print All Peterson Numbers from 1 to 500

Program

```
for num in range(1, 501):
    temp = num
    sum_fact = 0
    while temp > 0:
        digit = temp % 10
        fact = 1
        for i in range(1, digit + 1):
            fact *= i
```



```

sum_fact += fact
temp //= 10
if sum_fact == num:
    print(num, end=" ")

```

Output:- 1 2 145

Logic Explanation: Peterson Numbers from 1 to 500

Program Logic

- Loop runs from 1 to 500.
- For each number, factorial sum of digits is calculated.
- Numbers satisfying Peterson condition are printed.

Mathematical Logic

- Only numbers whose digit factorial sum equals the number itself are Peterson numbers.

108. Write a Program to Generate Pascal's Triangle

Program

```

n = int(input("Enter number of rows: "))
for i in range(n):
    num = 1
    print(" " * (n - i), end="")
    for j in range(i + 1):
        print(num, end=" ")
        num = num * (i - j) // (j + 1)
    print()

```

Sample Input 5

Output

```

1
1 1
1 2 1
1 3 3 1
1 4 6 4 1

```

Logic Explanation: Pascal's Triangle

Program Logic

- Number of rows is taken as input.
- Each row starts with 1.
- Each value is calculated using the previous value formula.
- Values are printed in triangular form.

Mathematical Logic

- Pascal's Triangle is based on **binomial coefficients**.
- Each element is:

$$C(n, r) = n! / (r! (n-r)!)$$

- Every number is the sum of the two numbers above it.

109. Write a Program to Find the Missing Number in a Given Sequence

Program

```

arr = list(map(int, input("Enter numbers separated by space: ").split()))
n = len(arr) + 1
total = n * (n + 1) // 2

print("Missing number:", total - sum(arr))

```

Sample Input

1 2 4 5

Output

Missing number: 3

Logic Explanation: Missing Number**Program Logic**

- A sequence of numbers is taken as input.
- Expected sum of numbers from 1 to n is calculated.
- Actual sum of given numbers is calculated.
- Difference gives the missing number.

Mathematical Logic

- Sum of first n natural numbers:

$$n(n + 1) / 2$$

- Missing number = Expected sum – Actual sum.

110. Write a Program to Check Whether a Number is a Fibonacci Number or Not**Program**

```
num = int(input("Enter a number: "))
a = 0
b = 1
while a < num:
    a, b = b, a + b
if a == num:
    print("Fibonacci Number")
else:
    print("Not a Fibonacci Number")
```

Sample Input: 21

Output

Fibonacci Number

Logic Explanation: Fibonacci Number**Program Logic**

- A number is taken as input.
- Fibonacci series is generated until the number is reached or exceeded.
- If the number appears in the series, it is Fibonacci.

Mathematical Logic

- Fibonacci series is defined as:

$$F(n) = F(n-1) + F(n-2)$$

- Series starts with 0 and 1.
- Example:

0, 1, 1, 2, 3, 5, 8, 13, 21

PATTERN PROGRAM

Pattern-1

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

Program:

```
n=int(input("Enter the number:"))
for i in range(1,n+1):
    print(" * "*n)
```

Pattern-2

```
1 2 3 4 5 6
1 2 3 4 5 6
1 2 3 4 5 6
1 2 3 4 5 6
1 2 3 4 5 6
1 2 3 4 5 6
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    for j in range(1, n+1):
        print(j, end=" ")
    print()
```

Pattern-3

```
1 1 1 1 1 1
2 2 2 2 2 2
3 3 3 3 3 3
4 4 4 4 4 4
5 5 5 5 5 5
6 6 6 6 6 6
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    for j in range(1, n+1):
        print(i, end=" ")
    print()
```

Pattern-4

```
A A A A A A
B B B B B B
C C C C C C
D D D D D D
E E E E E E
F F F F F F
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    for j in range(1, n+1):
        print(chr(64+i), end=" ")
    print()
```

Pattern-5

```
A B C D E F
A B C D E F
A B C D E F
A B C D E F
A B C D E F
A B C D E F
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    for j in range(1, n+1):
        print(chr(64+j), end=" ")
    print()
```

Pattern-6

```
a b c d e f
a b c d e f
a b c d e f
a b c d e f
a b c d e f
a b c d e f
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    for j in range(1, n+1):
        print(chr(96+j), end=" ")
    print()
```

Pattern-7

```
6 6 6 6 6 6
5 5 5 5 5 5
4 4 4 4 4 4
3 3 3 3 3 3
2 2 2 2 2 2
1 1 1 1 1 1
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    for j in range(1, n+1):
```

Pattern-8

```
6 5 4 3 2 1
6 5 4 3 2 1
6 5 4 3 2 1
6 5 4 3 2 1
6 5 4 3 2 1
6 5 4 3 2 1 58
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    for j in range(1, n+1):
```

Pattern-9

```
F F F F F F
E E E E E E
D D D D D D
C C C C C C
B B B B B B
A A A A A A
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    for j in range(1, n+1):
```



Pattern-10

```
F E D C B A
F E D C B A
F E D C B A
F E D C B A
F E D C B A
F E D C B A
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    for j in range(1, n+1):
        print(chr(65+n-j), end=" ")
    print()
```

Pattern-11

```
f f f f f f
e e e e e e
d d d d d d
c c c c c c
b b b b b b
a a a a a a
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    for j in range(1, n+1):
        print(chr(97+n-i), end=" ")
    print()
```

Pattern-12

```
f e d c b a
f e d c b a
f e d c b a
f e d c b a
f e d c b a
f e d c b a
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    for j in range(1, n+1):
        print(chr(97+n-j), end=" ")
    print()
```

Pattern-13

```
*
* *
* * *
* * * *
* * * * *
* * * * * *
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    for j in range(1, i+1):
        print("*", end=" ")
    print()
```

Pattern-14

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
6 6 6 6 6 6
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    for j in range(1, i+1):
        print(i, end=" ")
    print()
```

Pattern-15

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
1 2 3 4 5 6
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    for j in range(1, i+1):
        print(j, end=" ")
    print()
```

Pattern-16

```
A
B B
C C C
D D D D
E E E E E
F F F F F F
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    for j in range(1, n+1):
        print(n+1-i, end=" ")
    print()
```

Pattern-17

```
A
A B
A B C
A B C D
A B C D E
A B C D E F
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    for j in range(1, i+1):
        print(chr(64+j), end=" ")
    print()
```

Pattern-18

```
a
a b
a b c
a b c d
a b c d e
a b c d e f
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    for j in range(1, i+1):
        print(chr(96+j), end=" ")
    print()
```



Pattern-19

```
1 1 1 1 1 1
2 2 2 2 2
3 3 3 3
4 4 4
5 5
6
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    for j in range(1, n+2-i):
        print(i, end=" ")
    print()
```

Pattern-20

```
1 2 3 4 5 6
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
1
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    for j in range(1, n+2-i):
        print(j, end=" ")
    print()
```

Pattern-21

```
* * * * *
* * * * *
* * * *
* * *
* *
*
*
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    for j in range(1, n+2-i):
        print("*", end=" ")
    print()
```

Pattern-22

```
A A A A A A
B B B B B
C C C C
D D D
E E
F
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    for j in range(1, n+2-i):
        print(chr(64+i), end=" ")
    print()
```

Pattern-23

```
A B C D E F
A B C D E
A B C D
A B C
A B
A
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    for j in range(1, n+2-i):
        print(chr(64+j), end=" ")
    print()
```

Pattern-24

```
a b c d e f
a b c d e
a b c d
a b c
a b
a
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    for j in range(1, n+2-i):
        print(chr(96+j), end=" ")
    print()
```

Pattern-25

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
6 6 6 6 6 6
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    print(" "*(n-i), end="")
    for i in range(1, i+1):
```

Pattern-26

```
*
* *
* * *
* * * *
* * * * *
* * * * * *
```

Program:

```
60
n=int(input("Enter the number"))
for i in range(1,n+1):
    print(" "*(n-i), end="")
    for j in range(1, i+1):
```

Pattern-27

```
A
B B
C C C
D D D D
E E E E E
F F F F F F
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    print(" "*(n-i), end="")
    for i in range(1, i+1):
```

Pattern-28

```
6 6 6 6 6 6
5 5 5 5 5
4 4 4 4
3 3 3
2 2
1
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    for j in range(1, n+2-i):
        print(n+1-i, end=" ")
    print()
```

Pattern-29

```
F F F F F F
E E E E E
D D D D
C C C
B B
A
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    for j in range(1, n+2-i):
        print(chr(64+n+1-i), end=" ")
    print()
```

Pattern-30

```
f f f f f f
e e e e e
d d d d
c c c
b b
a
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    for j in range(1, n+2-i):
        print(chr(96+n+1-i), end=" ")
    print()
```

Pattern-31

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
6 6 6 6 6 6
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(1, i+1):
        print(i, end=" ")
    print()
```

Pattern-32

```
*
* *
* * *
* * * *
* * * * *
* * * * * *
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(1, i+1):
        print("*", end=" ")
    print()
```

Pattern-33

```
A
B B
C C C
D D D D
E E E E E
F F F F F F
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(1, i+1):
        print(chr(64+i), end=" ")
    print()
```

Pattern-34

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
1 2 3 4 5 6
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(1, i+1):
```

Pattern-35

```
a
a b
a b c
a b c d
a b c d e
a b c d e f
```

Program:

```
61
n=int(input("Enter the number"))
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(1, i+1):
```

Pattern-36

```
A
A B
A B C
A B C D
A B C D E
A B C D E F
```

Program:

```
n=int(input("Enter the number"))
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(1, i+1):
```



Pattern-37 <pre> ***** ***** **** *** ** *</pre> Program: <pre> n=int(input("Enter the number")) for i in range(1,n+1): print(" "*(i-1),"*"* (n+1-i))</pre>	Pattern-38 <pre> 666666 55555 4444 333 22 1</pre> Program: <pre> n=int(input("Enter the number")) for i in range(1,n+1): print(" "*(i-1),(str(n+1-i)+"")*(n+1-i))</pre>	Pattern-39 <pre> 111111 22222 3333 444 55 6</pre> Program: <pre> n=int(input("Enter the number")) for i in range(1,n+1): print(" "*(i-1),end="") for j in range(1,n+2-i): print(i,end="") print()</pre>
Pattern-40 <pre> 123456 12345 1234 123 12 1</pre> Program: <pre> n=int(input("Enter the number")) for i in range(1,n+1): print(" "*(i-1),end="") for j in range(1,n+2-i): print(j,end="") print()</pre>	Pattern-41 <pre> FFFFFF EEEE DDDD CCC BB A</pre> Program: <pre> n=int(input("Enter the number")) for i in range(1,n+1): print(" "*(i-1),(str(chr(65+n-i)) +"")*(n+1-i))</pre>	Pattern-42 <pre> ABCDEF ABCDE ABCD ABC AB A</pre> Program: <pre> n=int(input("Enter the number")) for i in range(1,n+1): print(" "*(i-1),end="") for j in range(65,66+n-i): print(chr(j),end="") print()</pre>
Pattern-43 <pre> * *** ***** ***** ***** ***** *****</pre> Program: <pre> n=int(input("Enter the number")) for i in range(1,n+1): print(" "*(n-i),"*"* (2*i-1))</pre>	Pattern-44 <pre> 1 222 33333 4444444 55555555 6666666666</pre> Program: <pre> n=int(input("Enter the number")) for i in range(1,n+1): print(" "*(n-i),(str(i)+"")*(2*i-1))</pre>	Pattern-45 <pre> 6 56 456 3456 23456 123456</pre> Program: <pre> n=int(input("Enter the number")) for i in range(1,n+1): print(" "*(n-i),end="") for j in range(1,i+1): print(j,end="") print()</pre>

<pre> Enter the number: 6 * ** *** **** ***** ***** Enter the number: 6 * ** *** **** ***** ***** Enter the number: 6 * ** *** **** ***** ***** </pre>	Pattern Program-46: <pre> n=int(input("Enter the number: ")) for i in range(1,n+1): print(" "*(n-i),end="") for j in range(1,i+1): print("*",end=" ") print() #.....other-1..... n=int(input("Enter the number: ")) for i in range(1,n+1): print(" "*(n-i),end="") for j in range(1,i+1): print("*",end=" ") for j in range(i-1,0,-1): print("*",end=" ") print() #.....other-2..... n=int(input("Enter the number: ")) for i in range(1,n+1): print(" "*(n-i),end="") for j in range(1,i+1): print("*",end=" ") for j in range(i-1,0,-1): print("*",end=" ") print() for i in range(n-1,0,-1): print(" "*(n-i),end="") for j in range(1,i+1): print("*",end=" ") for j in range(i-1,0,-1): print("*",end=" ") print() </pre>	Pattern Program-47: <pre> n=int(input("Enter the number: ")) for i in range(1,n+1): print(" "*(n-i),end="") for j in range(1,i+1): print(j,end=" ") print() #.....other-1..... n=int(input("Enter the number: ")) for i in range(1,n+1): print(" "*(n-i),end="") for j in range(1,i+1): print(j,end=" ") for j in range(i-1,0,-1): print(j,end=" ") print() #.....other-2..... n=int(input("Enter the number: ")) for i in range(1,n+1): print(" "*(n-i),end="") for j in range(1,i+1): print(j,end=" ") for j in range(i-1,0,-1): print(j,end=" ") print() for i in range(n-1,0,-1): print(" "*(n-i),end="") for j in range(1,i+1): print(j,end=" ") for j in range(i-1,0,-1): print(j,end=" ") print() </pre>
---	---	---

<pre> Enter the number: 6 1 1 2 1 1 2 3 2 1 1 2 3 4 3 2 1 1 2 3 4 5 4 3 2 1 </pre>	<pre> Enter the number: 6 1 1 2 1 1 2 3 2 1 1 2 3 4 3 2 1 </pre>	<pre> Enter the number: 6 1 1 2 1 2 3 </pre>
--	--	---




```

      1
    2 2 2
  3 3 3 3 3
4 4 4 4 4 4 4
5 5 5 5 5 5 5 5 5
6 6 6 6 6 6 6 6 6 6 6
  5 5 5 5 5 4 3 2 1
    4 4 4 4 3 2 1
      3 3 3 2 1
        2 2 1
          1

```

Pattern Program-48:

```

n=int(input("Enter the number: "))
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(1,i+1):
        print(i,end=" ")
    for j in range(i-1,0,-1):
        print(i,end=" ")
    print()
for i in range(n-1,0,-1):
    print(" "*(n-i),end="")
    for j in range(1,i+1):
        print(i,end=" ")
    for j in range(i-1,0,-1):
        print(j,end=" ")
    print()

```

```

      A
    A B A
  A B C B A
A B C D C B A
A B C D E D C B A
A B C D E F E D C B A
A B C D E D C B A
  A B C D C B A
    A B C B A
      A B A

```

Pattern Program-49:

```

n=int(input("Enter the number: "))
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(1,i+1):
        print(chr(64+j),end=" ")
    for j in range(i-1,0,-1):
        print(chr(64+j),end=" ")
    print()
for i in range(n-1,0,-1):
    print(" "*(n-i),end="")
    for j in range(1,i+1):
        print(chr(64+j),end=" ")
    for j in range(i-1,0,-1):
        print(chr(64+j),end=" ")
    print()

```

Pattern program-50

```
n=int(input("Enter the number: "))
for i in range(1,n+1):
    for j in range(1,i+1):
        if j==1 or j==i or i==n:
            print(j,end=" ")
        else:
            print(" ",end=" ")
    print()
#.....other-2.....
n=int(input("Enter the number: "))
for i in range(1,n+1):
    for j in range(1,i+1):
        if j==1 or j==i or i==n:
            print(i,end=" ")
        else:
            print(" ",end=" ")
    print()
#.....other-3.....
n=int(input("Enter the number: "))
for i in range(1,n+1):
    for j in range(1,i+1):
        if j==1 or j==i or i==n:
            print(chr(64+i),end=" ")
        else:
            print(" ",end=" ")
    print()
#.....other-4.....
n=int(input("Enter the number: "))
for i in range(1,n+1):
    for j in range(1,i+1):
        if j==1 or j==i or i==n:
            print(chr(96+j),end=" ")
        else:
            print(" ",end=" ")
    print()
```

Enter the number: 6
1
1 2
1 3
1 4
1 5
1 2 3 4 5 6

Enter the number: 6
1
2 2
3 3
4 4
5 5
6 6 6 6 6 6

Enter the number: 6
A
B B
C C
D D
E E
F F F F F F

Enter the number: 6
a
a b
a c
a d
a e
a b c d e f

Pattern Program-51

```
n=int(input("Enter the number: "))
for i in range(1,n+1):
    for j in range(1,n+1):
        print(min(i,j),end=" ")
    print()
#.....other-2.....
n=int(input("Enter the number: "))
for i in range(1,n+1):
    for j in range(1,n+1):
        print(min(i,j),end=" ")
    for j in range(n-1,0,-1):
        print(min(i,j),end=" ")
    print()
#.....other-2.....
n=int(input("Enter the number: "))
for i in range(1,n+1):
    for j in range(1,n+1):
        print(min(i,j),end=" ")
    for j in range(n-1,0,-1):
        print(min(i,j),end=" ")
    print()
for i in range(n-1,0,-1):
    for j in range(1,n+1):
        print(min(i,j),end=" ")
    for j in range(n-1,0,-1):
        print(min(i,j),end=" ")
    print()
#.....other.....
n=int(input("Enter the number: "))
for i in range(1,n+1):
    for j in range(1,n+1):
        print(min(chr(64+j),chr(64+i)),end=" ")
    for j in range(n-1,0,-1):
        print(min(chr(64+j),chr(64+i)),end=" ")
    print()
for i in range(n-1,0,-1):
    for j in range(1,n+1):
        print(min(chr(64+j),chr(64+i)),end=" ")
    for j in range(n-1,0,-1):
        print(min(chr(64+j),chr(64+i)),end=" ")
    print()
```

Enter the number: 6
1 1 1 1 1 1
1 2 2 2 2 2
1 2 3 3 3 3
1 2 3 4 4 4
1 2 3 4 5 5
1 2 3 4 5 6

Enter the number: 6
1 1 1 1 1 1 1 1 1 1
1 2 2 2 2 2 2 2 2 1
1 2 3 3 3 3 3 3 2 1
1 2 3 4 4 4 4 4 3 2 1
1 2 3 4 5 5 5 4 3 2 1
1 2 3 4 5 6 5 4 3 2 1

Enter the number: 6
1 1 1 1 1 1 1 1 1 1
1 2 2 2 2 2 2 2 2 1
1 2 3 3 3 3 3 3 2 1
1 2 3 4 4 4 4 4 3 2 1
1 2 3 4 5 5 5 4 3 2 1
1 2 3 4 5 6 5 4 3 2 1
1 2 3 4 5 5 5 4 3 2 1
1 2 3 4 4 4 4 4 3 2 1
1 2 3 3 3 3 3 3 2 1
1 2 2 2 2 2 2 2 2 1
1 1 1 1 1 1 1 1 1 1

Enter the number: 6
A A A A A A A A A A
A B B B B B B B B A
A B C C C C C C C B A
A B C D D D D D C B A
A B C D E E E D C B A
A B C D E F E D C B A
A B C D E E E D C B A
A B C D D D D D C B A
A B C C C C C C C B A
A B B B B B B B B A
A A A A A A A A A A

Pattern Program-52:

```
n=int(input("Enter the number: "))
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(1,i+1):
        if j==1:
            print("*",end=" ")
        else:
            print(" ",end=" ")
    print()#.....other-2.....
n=int(input("Enter the number: "))
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(1,i+1):
        if j==1:
            print("*",end=" ")
        else:
            print(" ",end=" ")
    print() #.....other-3.....
n=int(input("Enter the number: "))
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(1,i+1):
        if j==1:
            print("*",end=" ")
        else:
            print(" ",end=" ")
    print()
for i in range(n-1,0,-1):
    print(" "*(n-i),end="")
    for j in range(1,i+1):
        if j==1:
            print("*",end=" ")
        else:
            print(" ",end=" ")
    print()
```

Enter the number: 6

```
      *
     *
    *
   *
  *
 *
*
```

Enter the number: 6

```
      *
     * *
    * *
   * *
  * *
 * *
*
```

Enter the number: 6

```
      *
     * *
    * *
   * *
  * *
 * *
* *
*

```

Enter the number: 6

```
      1
     1 1
    1 1 1
   1 1 1 1
  1 1 1 1 1
 1 1 1 1 1 1
1 1 1 1 1 1 1

```

Enter the number: 6

```
      1
     1 1
    1 1 1
   1 1 1 1
  1 1 1 1 1
 1 1 1 1 1 1
1 1 1 1 1 1 1

```

Pattern Program-53:

```
n=int(input("Enter the number: "))
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(1,i+1):
        if j==1:
            print(j,end=" ")
        else:
            print(" ",end=" ")
    print() #.....other-5.....
n=int(input("Enter the number: "))
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(1,i+1):
        if j==1:
            print(j,end=" ")
        else:
            print(" ",end=" ")
    print() #.....other-6.....
n=int(input("Enter the number: "))
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(1,i+1):
        if j==1:
            print(j,end=" ")
        else:
            print(" ",end=" ")
    print()
for i in range(n-1,0,-1):
    print(" "*(n-i),end="")
    for j in range(1,i+1):
        if j==1:
            print(j,end=" ")
        else:
            print(" ",end=" ")
    print()
```

```
print(" ",end=" ")
for j in range(i-1,0,-1):
```



Pattern Program-56:

```
n=int(input("Enter the number: "))
c=1
for i in range(1,n+1):
    for j in range(1,i+1):
        print("{:2d}".format(c),end=" ")
        c+=1
    print()
n=int(input("Enter the number: "))
c=1
for i in range(1,n+1):
    for j in range(1,i+1):
        print("{:02d}".format(c),end=" ")
        c+=1
    print()

.....o/p.....
Enter the number: 6
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
6 6 6 6 6 6
Enter the number: 6
01
02 03
04 05 06
07 08 09 10
11 12 13 14 15
16 17 18 19 20 21
Enter the number: 6
```

Pattern Program-57:

```
n=int(input("Enter the number: "))
for i in range(1,n+1):
    c=i
    for j in range(1,n+1):
        print("{:3d}".format(c),end=" ")
        c=c+n
    print()
.....o/p.....
Enter the number: 6
1 7 13 19 25 31
2 8 14 20 26 32
3 9 15 21 27 33
4 10 16 22 28 34
5 11 17 23 29 35
6 12 18 24 30 36
***Our requirement is***
1
2
3
***program-1***
n=int(input("n="))
for i in range(1,n+1):
    print(i)
***Our requirement is***
* * *
* * *
* * *
***program-2***
n=int(input("n="))
for i in range(1,n+1):
    print("* " * n)
***Our requirement is***
1
2
3
1
2
3
68***Our requirement is***
1 1 1
2 2 2
3 3 3
```



Our requirement is

1 2 3

1 2 3

1 2 3

program-5

n=int(input("n="))

for i in range(1,n+1):

for j in range(1,n+1):

print(j,end=" ")

print()

Our requirement is

1 1 2 3

2 1 2 3

3 1 2 3

program-6

n=int(input("n="))

for i in range(1,n+1):

print(i,end=" ")

for j in range(1,n+1):

print(j,end=" ")

print()

Our requirement is

A A A A A A

B B B B B B

C C C C C C

D D D D D D

E E E E E E

F F F F F F

program-7

n=int(input("n="))

for i in range(1,n+1):

for j in range(1,n+1):

print(chr(64+i),end=" ")

#print(chr(96+i),end=" ")

print()

Our requirement is

a b c d e f

a b c d e f

a b c d e f

a b c d e f

a b c d e f

program-8

Our requirement is

6 6 6 6 6 6

5 5 5 5 5 5

4 4 4 4 4 4

3 3 3 3 3 3

2 2 2 2 2 2

1 1 1 1 1 1

program-8

n=int(input("n="))

for i in range(1,n+1):

for j in range(1,n+1):

print(n+1-i,end=" ")

print()

Our requirement is

6 5 4 3 2 1

6 5 4 3 2 1

6 5 4 3 2 1

6 5 4 3 2 1

6 5 4 3 2 1

6 5 4 3 2 1

program-8

n=int(input("n="))

for i in range(1,n+1):

for j in range(1,n+1):

print(n+1-j,end=" ")

print()

Our requirement is

F F F F F F

E E E E E E

D D D D D D

C C C C C C

B B B B B B

A A A A A A

program-9

n=int(input("n="))

for i in range(1,n+1):

for j in range(1,n+1):

print(chr(65+n-i),end=" ")

print()

Our requirement is

F E D C B A

F E D C B A

F E D C B A

F E D C B A

Our requirement is

1

222

33333

4444444

555555555

66666666666

program-36

n=int(input("n="))

for i in range(1,n+1):

print(" "*(n-i),(str(i)+"")*(2*i-1))

Our requirement is

A

BBB

CCCCC

DDDDDDD

EEEEEEEE

FFFFFFFFFFF

program-37

n=int(input("n="))

for i in range(1,n+1):

print(" "*(n-i),(str(chr(64+i)+""))*(2*i-1))

Our requirement is

A

CCC

EEEE

GGGGGGG

IIIIIIII

KKKKKKKKKK

program-38

n=int(input("n="))

for i in range(1,n+1):

print(" "*(n-i),(str(chr(64+2*i-1)+""))*(2*i-1))

Our requirement is

1

1 2 3

1 2 3 4 5

1 2 3 4 5 6 7

1 2 3 4 5 6 7 8 9

1 2 3 4 5 6 7 8 9 10 11

Our requirement is

```
1
3 2 1
5 4 3 2 1
7 6 5 4 3 2 1
9 8 7 6 5 4 3 2 1
11 10 9 8 7 6 5 4 3 2 1
```

program-40

```
n=int(input("n="))
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(2*i-1,0,-1):
        print(j,end=" ")
    print()
```

Our requirement is

```
A
A B C
A B C D E
A B C D E F G
A B C D E F G H I
A B C D E F G H I J K
```

program-41

```
n=int(input("n="))
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(65,65+2*i-1):
        print(chr(j),end=" ")
    print()
```

Our requirement is

```
A
C B A
E D C B A
G F E D C B A
I H G F E D C B A
K J I H G F E D C B A
```

program-42

```
n=int(input("n="))
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(65+2*i-2,64,-1):
        print(chr(j),end=" ")
    print()
```

Our requirement is

```
0
1 0 1
2 1 0 1 2
3 2 1 0 1 2 3
4 3 2 1 0 1 2 3 4
5 4 3 2 1 0 1 2 3 4 5
```

program-43

```
n=int(input("n="))
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(1,i):
        print(i-j,end=" ")
    for k in range(0,i):
        print(k,end=" ")
    print()
```

Our requirement is

```
A
B A B
C B A B C
D C B A B C D
E D C B A B C D E
F E D C B A B C D E F
```

program-44

```
n=int(input("n="))
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(1,i):
        print(chr(i-j+65),end=" ")
    for k in range(0,i):
        print(chr(k+65),end=" ")
    print()
```

Our requirement is

```
1
1 2 1
1 2 3 2 1
1 2 3 4 3 2 1
1 2 3 4 5 4 3 2 1
1 2 3 4 5 6 5 4 3 2 1
```

program-45

```
n=int(input("n="))
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(1,i+1):
```

Our requirement is

```
0
1 0 1
2 1 0 1 2
3 2 1 0 1 2 3
4 3 2 1 0 1 2 3 4
5 4 3 2 1 0 1 2 3 4 5
```

program-43

```
n=int(input("n="))
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(1,i):
        print(i-j,end=" ")
    for k in range(0,i):
        print(k,end=" ")
    print()
```

Our requirement is

```
A
B A B
C B A B C
D C B A B C D
E D C B A B C D E
F E D C B A B C D E F
```

program-44

```
n=int(input("n="))
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(1,i):
        print(chr(i-j+65),end=" ")
    for k in range(0,i):
        print(chr(k+65),end=" ")
    print()
```

Our requirement is

```
1
1 2 1
1 2 3 2 1
1 2 3 4 3 2 1
1 2 3 4 5 4 3 2 1
1 2 3 4 5 6 5 4 3 2 1
```

program-45

```
n=int(input("n="))
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(1,i+1):
```

Our requirement is

A
A B A
A B C B A
A B C D C B A
A B C D E D C B A
A B C D E F E D C B A

program-46

```
n=int(input("n="))
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(1,i+1):
        print(chr(64+j),end=" ")
    for k in range(i-1,0,-1):
        print(chr(64+k),end=" ")
    print()
```

Our requirement is

6
6 5
6 5 4
6 5 4 3
6 5 4 3 2
6 5 4 3 2 1

program-47

```
n=int(input("n="))
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(1,i+1):
        print(n+1-j,end=" ")
    print()
```

Our requirement is

* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *

program-48

```
n=int(input("n="))
for i in range(1,n+1):
    print(" "*(i-1),end="")
    for j in range(1,n+2-i):
        print("*",end=" ")
```

Our requirement is

6 6 6 6 6 6 6 6 6 6
5 5 5 5 5 5 5 5
4 4 4 4 4 4 4
3 3 3 3 3
2 2 2
1

program-49

```
n=int(input("n="))
for i in range(1,n+1):
    print(" "*(i-1),end="")
    for j in range(0,n+1-i):
        print(n+1-i,end=" ")
    for k in range(1,n+1-i):
        print(n+1-i,end=" ")
    print()
```

Our requirement is

9 9 9 9 9 9 9 9
7 7 7 7 7 7 7
5 5 5 5 5
3 3 3
1

program-50

```
n=int(input("n="))
for i in range(1,n+1):
    print(" "*(i-1),end="")
    for j in range(0,n+1-i):
        print(2*n+1-2*i,end=" ")
    for k in range(1,n+1-i):
        print(2*n+1-2*i,end=" ")
    print()
```

Our requirement is

1 2 3 4 5 6 7 8 9 10 11
1 2 3 4 5 6 7 8 9
1 2 3 4 5 6 7
1 2 3 4 5
1 2 3
1

program-51

```
n=int(input("n="))71
for i in range(1,n+1):
    print(" "*(i-1),end="")
    for j in range(1,n+2-i):
```

Our requirement is

F F F F F F F F F F
E E E E E E E E E
D D D D D D D D
C C C C C
B B B
A

program-52

```
n=int(input("n="))
for i in range(1,n+1):
    print(" "*(i-1),end="")
    for j in range(1,n+2-i):
        print(chr(65+n-i),end=" ")
    for k in range(2,n+2-i):
        print(chr(65+n-i),end=" ")
    print()
```

Our requirement is

K K K K K K K K K K
I I I I I I I I
G G G G G G G
E E E E E
C C C

A ***program-53***

```
n=int(input("n="))
for i in range(1,n+1):
    print(" "*(i-1),end="")
    for j in range(1,n+2-i):
        print(chr(65+2*n-2*i),end=" ")
    for k in range(2,n+2-i):
        print(chr(65+2*n-2*i),end=" ")
    print()
```

Our requirement is

1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
1 2 3 4 5 6
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
1

program-55

```
n=int(input("n="))
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(1,i+1):
```


*
**

*
1
22
333
4444
55555
6666666
77777777
888888888
7777777
6666666
55555
4444
333
22
1
1
12
123
1234
12345
123456
1234567
12345678
2345678
345678
45678
5678
678
78
8
1
12
123
1234
12345
123456
1234567
12345678
1234567
123456
12345
1234
123
12
1
A
BB
CCC
DDDD
EEEEEE
FFFFFFF
GGGGGGGG
HHHHHHHH
GGGGGGGG
FFFFFFF
EEEEEE
DDDD
CC C You Tube
B B

```

n=8
for i in range(1,n+1):
    print(" "*" (n-i),end="")
    for j in range(1,i+1):
        print(" *",end=" ")
    print()
for k in range(1,n):
    print(" "*"k,end="")
    for l in range(1,n+1-k):
        print(" *",end=" ")
    print()
for i in range(1,n+1):
    print(" "*" (n-i),end="")
    for j in range(1,i+1):
        print(i,end=" ")
    print()
for k in range(1,n):
    print(" "*"k,end="")
    for l in range(1,n+1-k):
        print(n-k,end=" ")
    print()
for i in range(1,n+1):
    print(" "*" (n-i),end="")
    for j in range(1,i+1):
        print(j,end=" ")
    print()
for k in range(1,n):
    print(" "*"k,end="")
    for l in range(1,n+1-k):
        print(k+l,end=" ")
    print()
for i in range(1,n+1):
    print(" "*" (n-i),end="")
    for j in range(1,i+1):
        print(j,end=" ")
    print()
for k in range(1,n):
    print(" "*"k,end="")
    for l in range(1,n+1-k):
        print(l,end=" ")
    print()
for i in range(1,n+1):
    print(" "*" (n-i),end="")
    for j in range(1,i+1):
        print(chr(64+i),end=" ")
    print()
for k in range(1,n):
    print(" "*"k,end="")
    for l in range(1,n+1-k):
        print(chr(64+n-k),end=" ")
    print()
for i in range(1,n+1):

```

```

*****
enter a number: 6
      *          *
    **         **
   ***        ***
  ****       ****
 *****     *****
*****

***program-87***
n=int(input("enter a number: "))
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(1,i+1):
        print("*",end=" ")
    print(" "*(n-i),end="")
    for k in range(1,i+1):
        print("*",end=" ")
    print()
...our requirements....
enter a number: 6
1
0 1
1 0 1
0 1 0 1
1 0 1 0 1
0 1 0 1 0 1

***program-88***
n=int(input("enter a number: "))
for i in range(1,n+1):
    for j in range(1,i+1):
        if (i%2!=0 and j%2!=0)or(i%2==0 and j%2==0):
            print("1",end=" ")
        else:
            print("0",end=" ")
    print()
...our requirements....
enter a number: 6
      *
     **
    ***
   ****
  *****
 *****
  *****
   ****
    ***
     **
      *
*****

```

*******program-89*****

```

n=int(input("enter a number: 
for a in range(1,n+1,2):
    for i in range(1,n+1):
        print(" "*(2*n-i-a),end="")
    for j in range(1,a+1):
        print("*",end=" ")
    print()
for b in range(1,n+1):
    print(" "*(n-2),end="")

```

```
n=int(input("enter a number: "))
for a in range(1,n+1,2):
    for i in range(1,n+1):
        print(" "*(2*n-i-a),end="")
        for j in range(1,i+a):
            print("*",end=" ")
        print()
    for b in range(1,n+1):
        print(" "*(n-2),end="")
        print("*"*2)
```



1. Logic Explanation: Find Minimum and Maximum Value from a List

Program:

```
l=[2,6,5,2,5,5,2,5,6,6,2,5,6,2,9,5,52,8,45,2,6,6,5,2]
a=l
min=max=l[0]
for i in l:
    if i<min:
        min=i
    elif i>max:
        max=i
print("minimum value=",min)
print("maximum value=",max)
```

Program:

```
l=[2,6,5,2,5,5,2,5,6,6,2,5,6,2,9,5,52,8,45,2,6,6,5,2]
l1,l2=min(l),max(l)
print("min=",l1,"and\t""max=",l2)
```

Output:

min= 2 and max= 52

Output:

```
minimum value= 2
maximum value= 52
....or by using predefined function...
```

Program Logic:

- A list of elements is stored in variable l.
- The first element of the list is assigned to both min and max.
- A loop traverses each element in the list.
- If the current element is smaller than min, the value of min is updated.
- If the current element is greater than max, the value of max is updated.
- After checking all elements, minimum and maximum values are displayed.

Mathematical Logic:

- The minimum value is the smallest number in the list.
- The maximum value is the largest number in the list.
- The comparison operators < and > are used to compare elements.
- By continuously updating values, the smallest and largest elements are obtained.

2. Logic Explanation: Rotate or Shift k Elements from Left to Right

Program:

```
l=list(map(int,input("enter the list elements: ").split()))
k=int(input("enter the value that you want to check how many times are repeated: "))
k=k%len(l)
```

```

for i in range(k):
    l.append(l.pop(i))
print(l)

```

Output:

enter the list elements: 5 6 8 63 5 2 5 4 2 5 4 2 5 4
 enter the value that you want to check how many times are repeated: 3
 [6, 63, 2, 5, 4, 2, 5, 4, 2, 5, 4, 5, 8, 5]

Program Logic:

- The list elements are taken as input from the user.
- The value of k is taken as input for rotation.
- $k \% \text{len}(l)$ is used to avoid extra rotations greater than list size.
- A loop runs k times.
- In each iteration, one element is removed using pop(i) and added at the end using append().
- Finally, the rotated list is displayed.

Mathematical Logic:

- List rotation shifts elements circularly from one position to another.
- The modulus operator % ensures valid rotation count.
- Each rotation moves one element from left side to right side.

3. Logic Explanation: Display Unique Vowels Present in a Word

Program:

```

w=input("enter the word to search for vowels: ")
vowels=['a','e','i','o','u']
v=[]
for i in w:
    if i in vowels:
        if i not in v:
            v.append(i)
print("vowels present in given word=",v)

```

Output:

enter the word to search for vowels: t3 skill center
 vowels present in given word= ['i', 'e']

Program Logic:

- A word is taken as input from the user.
- A list of vowels (a, e, i, o, u) is created.
- An empty list v is used to store unique vowels.
- A loop checks each character of the word.
- If the character is a vowel and not already present in v, it is added to the list.
- Finally, unique vowels are displayed.

Mathematical Logic:

- Vowels are selected using membership checking (in).
- Duplicate vowels are avoided by checking existing values before insertion.
- The final list contains only distinct vowel characters.

4. Logic Explanation: Search Particular Element Without Using Predefined Function

Program:

```
l=list(map(int,input("enter the list elements: ").split()))
k=int(input("enter the element that you want to search"))
for i in range(len(l)):
    if l[i]==k:
        print(i)
        break
    else:
        print('-1')
```

Output:

```
enter the list elements: 5 6 2 68 2 5 2 2 8 4 5 5 2 2
enter the element that you want to search6
```

Program Logic:

- The list elements are taken as input from the user.
- The element to be searched is stored in variable k.
- A loop traverses the list using index positions.
- If the element at current index matches k, the index is printed.
- The break statement stops the loop after first occurrence.
- If the element is not found, -1 is displayed.

Mathematical Logic:

- Linear searching compares the target element with each list element one by one.
- The equality operator (==) is used for comparison.
- If match occurs, the corresponding index position is returned.

5. Logic Explanation: Remove Duplicate Elements from a List

Program:

```
l=list(map(int,input("enter the list elements: ").split()))
print(list(set(l)))
```

Output:

```
enter the list elements: 5 6 3 87 5 24 5 2 54 5 2
[2, 3, 5, 6, 54, 87, 24]
```

2nd method:-

delete the duplicate element maintain the insertion order

```
l=list(map(int, input("enter the list elements: ").split()))
r=[]
for i in l:
    if i not in r:
        r.append(i)
print(r)
```

Output:

```
enter the list elements: 5 6 3 87 5 24 5 2 54 5 2
```

[5, 6, 3, 87, 24, 2, 54]

Program Logic:

- The list elements are taken as input from the user.
- In first method, set() removes duplicate values automatically.
- In second method, an empty list r is created.
- A loop checks each element of original list.
- If element is not present in r, it is appended.
- The final list contains only unique elements while maintaining insertion order.

Mathematical Logic:

- Duplicate elements are repeated values in a list.
- Set data structure stores only unique values.
- Membership checking (not in) avoids repeated insertion.

6. Logic Explanation: Count How Many Times an Element is Repeated

Program:

without predefined function.....

```
l=list(map(int,input("enter the list elements: ").split()))
k=int(input("enter the element that you want to search: "))
print(l.count(k))
```

Output:

```
enter the list elements: 5 6 3 87 5 24 5 2 54 5 2 5 2
enter the element that you want to search: 2
3
```

without predefined function....

Program:

```
l=list(map(int,input("enter the list elements: ").split()))
k=int(input())
c=0
for i in range(len(l)):
    if l[i]==k:
        c+=1
print(c)
```

Output: enter the list elements: 5 6 3 87 5 24 5 2 54 5 2 5 2

2
3

Program Logic:

- The list elements are taken as input from the user.
- The target element is stored in variable k.
- A counter variable c is initialized with value 0.
- A loop traverses all elements of the list.
- If the current element matches k, counter value increases by 1.
- Finally, the total count is displayed.

Mathematical Logic:

- Frequency means number of occurrences of an element.
- The equality operator (==) compares each element with k.
- Every successful match increases the frequency count by 1.

7. Logic Explanation: Find Maximum Repeated Element in a List

Program:

```
l=list(map(int, input("enter the list elements: ").split()))
e=0
m=0
for i in set(l):
    c=l.count(i)
    if c>m:
        m=c
        e=i
print("element=",e)
print("maximum coount=",m)
```

Output:

```
enter the list elements: 5 6 3 8 7 5 2 4 5 2 5 4 5 2 5 2
element= 5
maximum count= 5
```

Program Logic:

- The list elements are taken as input from the user.
- Variables m and e store maximum count and corresponding element.
- A loop checks every unique element of the list.
- count() calculates how many times the element appears.
- If current count is greater than maximum count, values are updated.
- Finally, the most repeated element and its count are displayed.

Mathematical Logic:

- Frequency of each element is calculated.
- The element with highest frequency is selected.
- The comparison operator (>) identifies the largest count value.

8. Logic Explanation: Perform Slicing Operation on a List

Program:

```
l=[2,4,54,5,67,7,8,99,9]
print(l[:])
print(l[:1])
print(l[:-1])
print(l[5])
print(l[:2])
print(l[5:2])
print(l[2:5:-1])
print(l[-4:])
print(l[-1])
print(l[-5:0:1])
```

```
print(l[-6:-1:1])
```

Output:

```
[2, 4, 54, 5, 67, 7, 8, 99, 9]
[2, 4, 54, 5, 67, 7, 8, 99, 9]
[9, 99, 8, 7, 67, 5, 54, 4, 2]
[2, 4, 54, 5, 67]
[2, 54, 67, 8, 9]
[]
[]
[7, 8, 99, 9]
9
[]
[5, 67, 7, 8, 99]
```

Program Logic:

- A list is created with multiple elements.
- Different slicing operations are performed using index positions.
- [::] prints the complete list.
- [::-1] reverses the list.
- [:5] prints first five elements.
- [::2] prints alternate elements.
- Negative indexing is also used for backward access.

Mathematical Logic:

- Slicing follows the format [start:end:step].
- Start index defines beginning position.
- End index defines stopping position.
- Step value controls interval between elements.
- Negative step reverses the traversal direction.

9. Logic Explanation: Shift First k Elements from Left to Right

Program:

```
l=list(map(int,input("enter the list elements: ").split()))
k=int(input())
k=k%len(l)
l=l[k:]+l[:k]
print(l)
```

Output:

```
enter the list elements: 6 3 5 8 9 5 4 2 5 4 52 4
3
[8, 9, 5, 4, 2, 5, 4, 52, 4, 6, 3, 5]
```

Program Logic:

- The list elements are taken as input from the user.
- The value of k is taken for shifting operation.
- k % len(l) ensures rotation remains within list size.

- List slicing divides the list into two parts.
- `l[k:]` contains remaining elements.
- `l[:k]` contains first k elements.
- Both parts are combined in shifted order.

Mathematical Logic:

- Rotation rearranges element positions circularly.
- Slicing separates the list into sections.
- Concatenation joins shifted parts to form final rotated list.

10. Logic Explanation: Sort First k Elements Ascending and Remaining Elements Descending

Program:

```
l=list(map(int, input("enter the list elements: ").split()))
k=int(input())
l=sorted(l[:k])+sorted(l[k:],reverse=True)
print(l)
```

Output:

```
enter the list elements: 5 8 7 2 3 1 4 96 45 78 5 45 9 86
3
[5, 7, 8, 96, 86, 78, 45, 45, 9, 5, 4, 3, 2, 1]
```

Program Logic:

- The list elements are taken as input from the user.
- The value of k is taken for dividing the list.
- The first k elements are selected using slicing.
- These elements are sorted in ascending order.
- The remaining elements are sorted in descending order using `reverse=True`.
- Both sorted parts are combined and displayed.

Mathematical Logic:

- Ascending order arranges elements from smaller to larger values.
- Descending order arranges elements from larger to smaller values.
- Slicing divides the list into two independent sections.
- The final list is created by concatenating both sorted sections.

10. Sort the First k Elements in Ascending Order and Remaining Elements in Descending Order

Program:

```
l=list(map(int,input("enter the list elements: ").split()))
k=int(input("enter k value: "))
l=sorted(l[:k])+sorted(l[k:],reverse=True)
print(l)
```

Output:


```
enter the list elements: 5 8 7 2 3 1 4 96 45 78
enter k value: 3
[5, 7, 8, 96, 78, 45, 4, 3, 2, 1]
```

Program Logic:

- Take list elements from the user.
- Take k value from the user.
- First k elements are sorted in ascending order.
- Remaining elements are sorted in descending order.
- Both parts are joined and printed.

Mathematical Logic:

- Ascending order arranges smaller to bigger numbers.
- Descending order arranges bigger to smaller numbers.
- List slicing divides the list into two parts.

11. Shift All Zeros to the End of the List**Program:**

```
l=list(map(int,input("enter the list elements: ").split()))
a=[]
for i in l:
    if i!=0:
        a.append(i)
c=l.count(0)
a.extend([0]*c)
print(a)
```

Output:

```
enter the list elements: 0 2 0 1 0 1 0 1 20 1
[2, 1, 1, 1, 20, 1, 0, 0, 0, 0]
```

Program Logic:

- Take list elements from the user.
- Store all non-zero elements in another list.
- Count total zeros in the list.
- Add all zeros at the end of the new list.
- Print the updated list.

Mathematical Logic:

- Zero elements are separated from non-zero elements.
- Count function finds total number of zeros.
- Extend function adds zeros at the end.

12. Find the Element Which Repeats Exactly One Time**Program:**

```
l=list(map(int,input("enter the list elements: ").split()))
for i in set(l):
    if l.count(i)==1:
        print(i)
        break
```

Output:

```
enter the list elements: 2 3 4 2 3 4 5
5
```

Program Logic:

- Take list elements from the user.
- Convert the list into a set to remove duplicates.
- Check count of each element.
- Print the element whose count is 1.

Mathematical Logic:

- Repeated elements have count greater than 1.
- Unique element has count equal to 1.

13. Bubble Sort Program**Program:**

```
l=list(map(int,input("enter the list elements: ").split()))
for i in range(len(l)-1):
    for j in range(i+1,len(l)):
        if l[j]<l[i]:
            l[i],l[j]=l[j],l[i]
print(l)
```

Output:

```
enter the list elements: 5 8 6 5 2 3 1 4 5
[1, 2, 3, 4, 5, 5, 6, 8]
```

Program Logic:

- Take list elements from the user.
- Compare elements one by one.
- Swap elements if left element is bigger.
- Repeat the process until list becomes sorted.

Mathematical Logic:

- Smaller elements move towards the beginning.
- Larger elements move towards the end after every pass.

14. Selection Sort Program**Program:**

```
l=list(map(int,input("enter the list elements: ").split()))
for i in range(len(l)-1):
    t=i
    for j in range(i+1,len(l)):
        if l[j]<l[t]:
            t=j
    l[i],l[t]=l[t],l[i]
print(l)
```

Output:

```
enter the list elements: 5 8 6 5 2 3 1 4 5
[1, 2, 3, 4, 5, 5, 6, 8]
```

Program Logic:

- Take list elements from the user.
- Find the smallest element in every iteration.
- Swap it with the current position.
- Repeat until list becomes sorted.

Mathematical Logic:

- Minimum element is selected in each pass.
- Swapping places the minimum element at correct position.

15. Take Input as 2-D List

Program:

```
l=[]
x=input("enter the elements: ").split(':')
for i in x:
    l.append(list(map(int,i.split()))))
print(l)
```

Output:

```
enter the elements: 1 2 3:4 5 6:7 8 9
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

Program Logic:

- Take all rows as input separated by colon (:).
- Split the rows using split(':').
- Convert each row into list using split().
- Store all rows inside another list.

Mathematical Logic:

- Each inner list represents one row.
- Nested lists create a 2-D matrix structure.

16. Sort First k Elements Ascending and Remaining Elements Descending

Program:

```
l=list(map(int,input("enter elements: ").split()))
k=int(input("enter k value: "))
l=sorted(l[:k])+sorted(l[k:],reverse=True)
print(l)
```

Output:

```
enter elements: 2 8 3 4 6 1 0 5 7 9
enter k value: 3
[2, 3, 8, 9, 7, 6, 5, 4, 1, 0]
```

Program Logic:

- Take list elements from the user.
- Divide the list using k value.
- Sort first part in ascending order.
- Sort remaining part in descending order.
- Combine and print the final list.

Mathematical Logic:

- Ascending order arranges smaller to larger values.
- Descending order arranges larger to smaller values.

17. Shift All Zeros to the End of the List

Program:

```
L=[0,0,3,1,0,3,0,12,0,0,7]
c=L.count(0)
L=[i for i in L if i!=0]
L.extend([0]*c)
print(L)
```

Output:

```
[3, 1, 3, 12, 7, 0, 0, 0, 0, 0, 0]
```

Program Logic:

- Count all zero elements in the list.
- Remove all zeros using list comprehension.
- Add zeros at the end using extend().
- Print the updated list.

Mathematical Logic:

- Zero count helps determine how many zeros to append.
- Non-zero elements keep their original order.

18. Find the Element Which Repeats Exactly One Time

Program:

```
l=list(map(int,input("enter elements: ").split()))
for i in set(l):
    if l.count(i)==1:
        print(i)
```

Output:

```
enter elements: 1 2 3 2 1 4 3
4
```

Program Logic:

- Take list elements from the user.
- Remove duplicate values using set().
- Check frequency of every element.
- Print the element having count 1.

Mathematical Logic:

- Frequency means number of occurrences of an element.
- Unique element appears only once.

19. Find the Maximum Number in 2-D List

Program:

```
l=[]
n=int(input("enter n value: "))
for i in range(n):
    l.append(list(map(int,input("enter element: ").split())))
m=l[0][0]
for row in l:
    for element in row:
        if element>m:
            m=element
print("list=",l)
print("max=",m)
```

Output:

```
enter n value: 3
enter element: 2 5 8
enter element: 9 1 4
enter element: 7 6 3
list= [[2, 5, 8], [9, 1, 4], [7, 6, 3]]
```

max= 9

Program Logic:

- Take rows of 2-D list from the user.
- Assume first element as maximum.
- Traverse all rows and elements.
- Compare each element with maximum value.
- Print the largest element.

Mathematical Logic:

- Comparison operation finds the greatest value.
- Nested loops access every element of matrix.

20. Find the Maximum Number from List Inside List

Program:

```
l=[2,3,5,6,[3,5,6,7,39],[9,6,5],7,5]
m=0
for i in l:
    if type(i)==list:
        for j in i:
            if j>m:
                m=j
    else:
        if i>m:
            m=i
```

```
print("list=",l)
print("max=",m)
```

Output:

```
list= [2, 3, 5, 6, [3, 5, 6, 7, 39], [9, 6, 5], 7, 5]
max= 39
```

Program Logic:

- Create a list containing numbers and sublists.
- Check whether element is a list or normal number.
- Traverse sublists separately.
- Compare all elements to find maximum value.
- Print the maximum element.

Mathematical Logic:

- Nested loops help access elements inside sublists.
- Comparison operation identifies the largest number.

21. Reverse the Given List Without Using Predefined Function

Program:

```
l=list(map(int,input("enter the list elements: ").split()))
a=[]
for i in range(len(l)-1,-1,-1):
    a.append(l[i])
print(a)
```

Output:

```
enter the list elements: 1 2 3 4 5
[5, 4, 3, 2, 1]
```

Program Logic:

- Take list elements from the user.
- Traverse the list from last index to first index.
- Append each element into a new list.
- Print the reversed list.

Mathematical Logic:

- Negative indexing helps access elements in reverse order.
- Loop decreases index step by step from end to start.

22. Merge Two Lists into a Single List**Program:**

```
l1=list(map(int,input("enter first list elements: ").split()))
l2=list(map(int,input("enter second list elements: ").split()))
l=l1+l2
print(l)
```

Output:

```
enter first list elements: 1 2 3
enter second list elements: 4 5 6
[1, 2, 3, 4, 5, 6]
```

Program Logic:

- Take two lists from the user.
- Use + operator to combine both lists.
- Store merged list in another variable.
- Print the final merged list.

Mathematical Logic:

- Concatenation joins two sequences into one sequence.

23. Find the Second Largest Element from the Given List**Program:**

```
l=list(map(int,input("enter the list elements: ").split()))
l=sorted(set(l))
print("second largest element is:",l[-2])
```

Output:

```
enter the list elements: 5 8 2 9 6 1
second largest element is: 8
```

Program Logic:

- Take list elements from the user.
- Remove duplicate values using set().
- Sort the list in ascending order.
- Access second last element using negative index.
- Print the second largest element.

Mathematical Logic:

- Largest element is at the last position after sorting.
- Second largest element is placed before the last element.

24. Find the Second Smallest Element from the Given List**Program:**

```
l=list(map(int,input("enter the list elements: ").split()))
l=sorted(set(l))
```

```
print("second smallest element is:",l[1])
```

Output:

```
enter the list elements: 5 8 2 9 6 1
second smallest element is: 2
```

Program Logic:

- Take list elements from the user.
- Remove duplicate elements using set().
- Sort the list in ascending order.
- Print the second element from the sorted list.

Mathematical Logic:

- Smallest element comes first after sorting.
- Second smallest element is placed at index 1.

25. Count Even and Odd Numbers Present in the List

Program:

```
l=list(map(int,input("enter the list elements: ").split()))

even=0
odd=0
for i in l:
    if i%2==0:
        even=even+1
    else:
        odd=odd+1
print("even numbers =",even)
print("odd numbers =",odd)
```

Output:

```
enter the list elements: 1 2 3 4 5 6 7
even numbers = 3
odd numbers = 4
```

Program Logic:

- Take list elements from the user.
- Check every element using modulus operator.
- If remainder is 0, count as even.
- Otherwise count as odd.
- Print total even and odd numbers.

Mathematical Logic:

- Even number gives remainder 0 when divided by 2.
- Odd number gives remainder 1 when divided by 2.

26. Separate Positive and Negative Numbers from the Given List

Program:

```
l=list(map(int,input("enter the list elements: ").split()))
p=[]
n=[]
for i in l:
    if i>=0:
        p.append(i)
```

```

else:
    n.append(i)
print("positive numbers =",p)
print("negative numbers =",n)

```

Output:

```

enter the list elements: 5 -2 8 -1 0 -9 7
positive numbers = [5, 8, 0, 7]
negative numbers = [-2, -1, -9]

```

Program Logic:

- Take list elements from the user.
- Create separate lists for positive and negative numbers.
- Check each element using condition.
- Store numbers in their respective lists.
- Print both lists.

Mathematical Logic:

- Positive numbers are greater than or equal to 0.
- Negative numbers are less than 0.

27. Find the Sum of All Elements in the Given List

Program:

```

l=list(map(int,input("enter the list elements: ").split()))
s=0
for i in l:
    s=s+i
print("sum =",s)

```

Output:

```

enter the list elements: 1 2 3 4 5
sum = 15

```

Program Logic:

- Take list elements from the user.
- Initialize sum variable with 0.
- Add every element to the sum variable.
- Print the final sum.

Mathematical Logic:

- Addition combines all values into one total value.

28. Find the Average of Elements Present in the List

Program:

```

l=list(map(int,input("enter the list elements: ").split()))
s=0
for i in l:
    s=s+i
avg=s/len(l)
print("average =",avg)

```

Output:

```

enter the list elements: 2 4 6 8
average = 5.0

```


Program Logic:

- Take list elements from the user.
- Find total sum of all elements.
- Divide sum by total number of elements.
- Print the average value.

Mathematical Logic:

- Average = Sum of elements ÷ Total number of elements.

29. Insert an Element at a Particular Position in the List**Program:**

```
l=list(map(int,input("enter the list elements: ").split()))
pos=int(input("enter position: "))
ele=int(input("enter element: "))
l.insert(pos,ele)

print(l)
```

Output:

```
enter the list elements: 1 2 3 4 5
enter position: 2
enter element: 10
[1, 2, 10, 3, 4, 5]
```

Program Logic:

- Take list elements from the user.
- Take position and new element as input.
- Use insert() function to add element at given position.
- Print the updated list.

Mathematical Logic:

- Insertion shifts remaining elements one position forward.

30. Delete an Element from a Particular Position in the List**Program:**

```
l=list(map(int,input("enter the list elements: ").split()))
pos=int(input("enter position: "))
del l[pos]
print(l)
```

Output:

```
enter the list elements: 1 2 3 4 5
enter position: 2
[1, 2, 4, 5]
```

Program Logic:

- Take list elements from the user.
- Take position value from the user.
- Delete element using del keyword.
- Print the updated list.

Mathematical Logic:

- Deletion removes the element from the specified index.
- Remaining elements shift one position backward.

31. Reverse Each Element Present in the Given List of Strings

Program:

```
l=input("enter string elements: ").split()
a=[]
for i in l:
    a.append(i[::-1])
print(a)
```

Output:

```
enter string elements: python java html
['nohtyp', 'avaj', 'lmth']
```

Program Logic:

- Take string elements from the user.
- Traverse each string one by one.
- Reverse every string using slicing.
- Store reversed strings in a new list.
- Print the final list.

Mathematical Logic:

- Slicing with -1 step reverses the string.
- Each character position is accessed from end to start.

32. Check Whether the Given List is Palindrome or Not**Program:**

```
l=list(map(int,input("enter the list elements: ").split()))
if l==l[::-1]:
    print("palindrome list")
else:
    print("not palindrome list")
```

Output:

```
enter the list elements: 1 2 3 2 1
palindrome list
```

Program Logic:

- Take list elements from the user.
- Reverse the list using slicing.
- Compare original list with reversed list.
- If both are same, list is palindrome.

Mathematical Logic:

- Palindrome sequence remains same after reversing.

33. Find the Common Elements Between Two Lists**Program:**

```
l1=list(map(int,input("enter first list: ").split()))
l2=list(map(int,input("enter second list: ").split()))
for i in l1:
    if i in l2:
        print(i,end=" ")
```

Output:

```
enter first list: 1 2 3 4 5
enter second list: 3 4 5 6 7
3 4 5
```

Program Logic:

- Take two lists from the user.
- Traverse elements of first list.
- Check whether element exists in second list.
- Print common elements.

Mathematical Logic:

- Common elements are present in both lists.
- Membership operator checks existence of element.

34. Find the Missing Number from the Given List

Program:

```
l=list(map(int,input("enter the list elements: ").split()))
n=max(l)
s=n*(n+1)//2
print("missing number =",s-sum(l))
```

Output:

```
enter the list elements: 1 2 3 5
missing number = 4
```

Program Logic:

- Take list elements from the user.
- Find maximum value of the list.
- Calculate sum of natural numbers.
- Subtract actual list sum from expected sum.
- Print missing number.

Mathematical Logic:

$$S=\frac{n(n+1)}{2}$$

- Formula gives sum of first n natural numbers.
- Difference between expected and actual sum gives missing number.

35. Convert All Negative Numbers into Positive Numbers in the List

Program:

```
l=list(map(int,input("enter the list elements: ").split()))
a=[]
for i in l:
    if i<0:
        a.append(-i)
    else:
        a.append(i)
print(a)
```

Output:

```
enter the list elements: 5 -2 8 -1 -9
[5, 2, 8, 1, 9]
```

Program Logic:

- Take list elements from the user.
- Check whether element is negative.
- Convert negative number into positive using minus sign.
- Store updated values in new list.
- Print the final list.

Mathematical Logic:

- Negative sign multiplication converts negative to positive.

36. Create a New List with Square of All Elements

Program:

```
l=list(map(int,input("enter the list elements: ").split()))
a=[]
for i in l:
    a.append(i*i)
print(a)
```

Output:

```
enter the list elements: 1 2 3 4
[1, 4, 9, 16]
```

Program Logic:

- Take list elements from the user.
- Traverse each element of the list.
- Multiply the element by itself.
- Store square values in new list.
- Print the squared list.

Mathematical Logic:

$a^2 = a \times a$

- Square means multiplication of a number by itself.

37. Find the Frequency of Each Element in the Given List

Program:

```
l=list(map(int,input("enter the list elements: ").split()))
for i in set(l):
    print(i,"=",l.count(i))
```

Output:

```
enter the list elements: 1 2 2 3 3 3 4
1 = 1
2 = 2
3 = 3
4 = 1
```

Program Logic:

- Take list elements from the user.
- Remove duplicate elements using set().
- Count occurrence of every element.
- Print frequency of each element.

Mathematical Logic:

- Frequency means total number of occurrences of an element.

38. Swap the First and Last Element of the Given List

Program:

```
l=list(map(int,input("enter the list elements: ").split()))
l[0],l[-1]=l[-1],l[0]
print(l)
```

Output:

```
enter the list elements: 1 2 3 4 5
```

[5, 2, 3, 4, 1]

Program Logic:

- Take list elements from the user.
- Swap first element with last element.
- Print the updated list.

Mathematical Logic:

- Index 0 represents first element.
- Index -1 represents last element.

39. Split the List into Two Equal Parts

Program:

```
l=list(map(int,input("enter the list elements: ").split()))
m=len(l)//2
a=l[:m]
b=l[m:]
print(a)
print(b)
```

Output:

```
enter the list elements: 1 2 3 4 5 6
[1, 2, 3]
[4, 5, 6]
```

Program Logic:

- Take list elements from the user.
- Find middle position using length of list.
- Divide list into two parts using slicing.
- Print both parts separately.

Mathematical Logic:

- Integer division gives middle index value.
- Slicing separates the list into equal halves.

40. Find the Largest Element in Each Row of a 2-D List

Program:

```
l=[]
n=int(input("enter number of rows: "))
for i in range(n):
    l.append(list(map(int,input("enter elements: ").split())))
for row in l:
    print("largest element =",max(row))
```

Output:

```
enter number of rows: 3
enter elements: 2 5 8
enter elements: 9 1 4
enter elements: 7 6 3
largest element = 8
largest element = 9
largest element = 7
```

Program Logic:

- Take rows of 2-D list from the user.
- Traverse each row one by one.

- Use max() function to find largest element of row.
- Print largest value of every row.

Mathematical Logic:

- Maximum value is the greatest element in a row.
- Each row is processed independently.

1. Reverse the Given String

Program:

```
s=input("Enter the string: ")
print(s[::-1])
```

Output:

```
Enter the string: skill
lliks
```

Program Logic:

- Take string input from the user.
- Reverse the string using slicing.
- Print the reversed string.

Mathematical Logic:

- Slicing with step -1 reverses the sequence from end to start.

2. Reverse the Order of Words

Program:

```
s=input("Enter the string: ")
l=s.split()
l1=[]
i=len(l)-1
while i>=0:
    l1.append(l[i])
    i=i-1
r=" ".join(l1)
print(r)
```

Output:

```
Enter the string: TWKSAA SKILLS CENTER
CENTER SKILLS TWKSAA
```

Program Logic:

- Take string input from the user.
- Split the string into words.
- Traverse the list from last word to first word.
- Join all words into a single string.
- Print the reversed word order.

Mathematical Logic:

- Words are rearranged from last position to first position.

3. Check Whether the Given String is Palindrome or Not

Program:

```
s=input("Enter the string: ")
if s==s[::-1]:
    print("palindrome")
```

```
else:  
    print("not palindrome")
```

Output:

```
Enter the string: madam  
palindrome
```

Program Logic:

- Take string input from the user.
- Reverse the string using slicing.
- Compare original string with reversed string.
- If both are same, print palindrome.

Mathematical Logic:

- Palindrome string remains same after reversing.

4. Reverse the Internal Content of Each Word

Program:

```
s=input("Enter the string: ")  
  
l=s.split()  
l1=[]  
for i in l:  
    l1.append(i[::-1])  
r=" ".join(l1)  
print(r)
```

Output:

```
Enter the string: twksaa skills center  
aaskwt slliks retneC
```

Program Logic:

- Take string input from the user.
- Split the string into words.
- Reverse every word individually.
- Join reversed words into a string.
- Print the result.

Mathematical Logic:

- Each word is reversed independently without changing word order.

5. Reverse the String Without Reversing the Special Symbols

Program:

```
l=list(input("Enter the string: "))  
i=0  
j=len(l)-1  
while i<j:  
    if not l[i].isalnum():  
        i=i+1  
    elif not l[j].isalnum():  
        j=j-1  
    else:  
        l[i],l[j]=l[j],l[i]  
        i=i+1
```

```
j=j-1
print("".join(l))
```

Output:

```
Enter the string: a@g#c%dgh@#ghg@
g@h#g%hgd@#cga@
```

Program Logic:

- Take string input from the user.
- Convert string into list for swapping.
- Ignore special symbols using isalnum().
- Swap only alphabet and numeric characters.
- Join the list into string and print it.

Mathematical Logic:

- Two pointer method swaps valid characters from both ends.
- Special symbols remain fixed at same positions.

6. Print Characters at Odd Position and Even Position

Program:

```
s=input("Enter the string: ")
print("characters at even position:",s[0::2])
print("characters at odd position:",s[1::2])
```

Output:

```
Enter the string: skills
characters at even position: sil
characters at odd position: kls
```

Program Logic:

- Take string input from the user.
- Use slicing to access even position characters.
- Use slicing to access odd position characters.
- Print both results separately.

Mathematical Logic:

- Step value 2 skips alternate characters.

7. Extract Numeric Characters and Form Largest Possible Even Number

Program:

```
s=input("Enter the string: ")
l=[]
for i in s:
    if i.isdecimal():
        l.append(i)
l.sort(reverse=True)
for i in range(len(l)-1,-1,-1):
    if int(l[i])%2==0:
        l.append(l.pop(i))
        print(int("".join(l)))
        break
else:
```



```
print(-1)
```

Output:

```
Enter the string: d4f6g5h7
7654
```

Program Logic:

- Take alphanumeric string from the user.
- Extract only numeric characters.
- Sort digits in descending order.
- Find an even digit and place it at the end.
- Print the largest possible even number.

Mathematical Logic:

- Even number must end with 0, 2, 4, 6, or 8.
- Descending arrangement gives maximum possible value.

8. Merge Characters of Two Strings Alternatively

Program:

```
s1=input("Enter first string: ")
s2=input("Enter second string: ")
r=""
i,j=0,0
while i<len(s1) or j<len(s2):
    if i<len(s1):
        r=r+s1[i]
        i=i+1
    if j<len(s2):
        r=r+s2[j]
        j=j+1
print(r)
```

Output:

```
Enter first string: raja
Enter second string: ram
rraajma
```

Program Logic:

- Take two strings from the user.
- Traverse both strings simultaneously.
- Add one character alternatively from each string.
- Print the merged string.

Mathematical Logic:

- Alternate arrangement combines characters one by one.

9. Return the Super Reduced String

Program:

```
s=list(input("Enter the string: "))
i=0
while i<len(s)-1:
```

```

    if s[i]==s[i+1]:
        del s[i]
        del s[i]
        i=0
    else:
        i=i+1
if len(s)==0:
    print("empty string")
else:
    print("".join(s))

```

Output:

Enter the string: abbcbbca
 abcbca

Program Logic:

- Take string input from the user.
- Compare adjacent characters.
- Delete adjacent equal characters.
- Continue until no equal adjacent characters remain.
- Print the reduced string.

Mathematical Logic:

- Adjacent duplicate pairs are removed repeatedly.

10. Sort Characters of String with Alphabets First and Numbers Later

Program:

```

s=input("Enter the string: ")
s1=""
s2=""
r=""
for i in s:
    if i.isalpha():
        s1=s1+i
    else:
        s2=s2+i
for i in sorted(s1):
    r=r+i
for i in sorted(s2):
    r=r+i
print(r)

```

Output:

Enter the string: B6C3D1E2A8
 ABCDE12368

Program Logic:

- Take string input from the user.
- Separate alphabets and numeric characters.
- Sort alphabets independently.
- Sort numeric characters independently.
- Join both sorted parts together.

Mathematical Logic:

- Alphabetic sorting arranges letters in dictionary order.
- Numeric sorting arranges digits in ascending order.

11. Check Whether Given String is Pangram or Not

Program:

```
s=input("Enter the string: ").lower()
a="abcdefghijklmnopqrstuvwxyz"
for i in a:
    if i not in s:
        print("not pangram")
        break
else:
    print("pangram")
```

Output:

Enter the string: the quick brown fox jumps over a lazy dog

pangram

Program Logic:

- Take string input from the user.
- Convert string into lowercase.
- Check whether all alphabets are present.
- If any alphabet is missing, print not pangram.
- Otherwise print pangram.

Mathematical Logic:

- Pangram sentence contains all 26 English alphabets at least once.

12. Check Whether Given String is Anagram or Not

Program:

```
s1=input("Enter first string: ")
s2=input("Enter second string: ")
l1=sorted(s1)
l2=sorted(s2)
if l1==l2:
    print("anagram")
else:
    print("not anagram")
```

Output:

Enter first string: abcd
Enter second string: bdac
anagram

Program Logic:

- Take two strings from the user.
- Sort both strings.
- Compare sorted strings.
- If both are same, strings are anagrams.

Mathematical Logic:

- Anagram strings contain same characters with same frequency.

13. Expand String Based on Given Number

Program:

```
s=input("Enter the string: ")
r=""
for i in range(0,len(s),2):
    r=r+s[i]*int(s[i+1])
print(r)
```

Output:

```
Enter the string: a3b6c2
aaabbbbbbbcc
```

Program Logic:

- Take string input from the user.
- Traverse characters and numbers together.
- Repeat alphabet according to given number.
- Store result in another string.
- Print the expanded string.

Mathematical Logic:

- Multiplication of string repeats character multiple times.

14. Perform the Following Activity

Program:

```
s=input("Enter the string: ")
r=""
for i in range(0,len(s),2):
    ch=s[i]
    n=int(s[i+1])
    r=r+ch+chr(ord(ch)+n)
print(r)
```

Output:

```
Enter the string: a4k3b2
aeknbd
```

Program Logic:

- Take string input from the user.
- Extract alphabet and number pair.
- Add original character to result.
- Add next character after shifting ASCII value.
- Print final string.

Mathematical Logic:

- ord() converts character into ASCII value.
- chr() converts ASCII value back into character.

Program:

```
s=input("Enter the string: ")
l=[]
for i in s:
    if i not in l:
        l.append(i)
r="".join(l)
print(r)
```

Output:

Enter the string: ABCDABACBEEDBCFBDBDCFCFCDFC
ABCDEF

Program Logic:

- Take string input from the user.
- Traverse each character one by one.
- Add character only if not already present.
- Join all unique characters into string.
- Print final result.

Mathematical Logic:

- Duplicate values are ignored after first occurrence.

16. Find the Number of Occurrences of Each Character

Program:

```
s=input("Enter the string: ")
d={}
for i in s:

    if i in d:
        d[i]=d[i]+1
    else:
        d[i]=1
for k,v in d.items():
    print(k,"=",v,"Times")
```

Output:

```
Enter the string: ABABSBADBDBABAB
A = 4 Times
B = 5 Times
S = 2 Times
D = 1 Times
```

Program Logic:

- Take string input from the user.
- Create empty dictionary.
- Count frequency of every character.
- Store character and count in dictionary.
- Print all character frequencies.

Mathematical Logic:

- Frequency means total occurrences of each character.

17. Reverse Alternate Words in the Sentence

Program:

```
s=input("Enter the sentence: ")
l=s.split()
for i in range(len(l)):
    if i%2!=0:
        l[i]=l[i][::-1]
print(" ".join(l))
```

Output:

```
Enter the sentence: one two three four five six seven
one owt three ruof five xis seven
```

Program Logic:

- Take sentence input from the user.
- Split sentence into words.
- Reverse words at odd positions.
- Join all words into a sentence.
- Print final output.

Mathematical Logic:

- Modulus operator identifies alternate positions.

18. Find the Character Weight of Given String**Program:**

```
s=input("Enter the string: ").lower()
sum=0
for i in s:
    sum=sum+(ord(i)-96)
print(sum)
```

Output: Enter the string: TWKSAA
81

Program Logic:

- Take string input from the user.
- Convert string into lowercase.
- Find alphabetical position of each character.
- Add all character weights together.
- Print total weight.

Mathematical Logic:

Weight=\sum (Alphabet\ Position)

- a=1, b=2, c=3 ... z=26.
- ASCII value helps calculate alphabet position.

19. Count Consecutive Characters in String**Program:**

```
s=input("Enter the string: ")
r=""
c=1
for i in range(len(s)-1):
    if s[i]==s[i+1]:
        c=c+1
    else:
        r=r+s[i]+str(c)
        c=1
r=r+s[-1]+str(c)
print(r)
```

Output:

Enter the string: aaabbcdcdce
a3b2c1d2c1d1e1

Program Logic:

- Take string input from the user.
- Count consecutive repeating characters.
- Store character with its count.

- Reset counter when character changes.
- Print compressed string.

Mathematical Logic:

- Consecutive frequency counts repeated adjacent characters.

20. Expand the Compressed String

Program:

```
s=input("Enter the string: ")
r=""
for i in range(0,len(s),2):
    r=r+s[i]*int(s[i+1])
print(r)
```

Output:

```
Enter the string: a3b2c1d3e1
aaabbccdde
```

Program Logic:

- Take compressed string from the user.
- Extract alphabet and count pair.
- Repeat alphabet according to count value.
- Join repeated characters into string.
- Print expanded string.

Mathematical Logic:

- String multiplication repeats characters multiple times.

21. Check How Many Times "rama" Can Be Formed

Program:

```
s=input("Enter the string: ")
r=min(s.count('r'),
      s.count('a')//2,
      s.count('m'))
print(r)
```

Output: Enter the string: rrmmaaarwxyz
1

Program Logic:

- Take string input from the user.
- Count occurrences of r, a, and m.
- Since "rama" contains two a's, divide count of a by 2.
- Find minimum count among required characters.
- Print total possible formations.

Mathematical Logic:

- Number of words formed depends on minimum available character count.

22. Find Edit Distance to Make String Palindrome

Program:

```
s=input("Enter the string: ")
d=dict(zip([chr(i) for i in range(ord('a'),ord('z')+1)],range(1,27)))
w=0
i=0
j=len(s)-1
while i<j:
    w=w+abs(d[s[i]]-d[s[j]])
```

```
i=i+1
j=j-1
print(w)
```

Output:

```
Enter the string: world
40
```

Program Logic:

- Take string input from the user.
- Assign weight values to alphabets.
- Compare characters from both ends.
- Find difference between character weights.
- Add all differences and print result.

Mathematical Logic:

Distance=|a-b|

- Absolute difference gives character edit distance.
- Total distance is sum of all pair differences.

23. Count Total Number of Vowels in the Given String

Program:

```
s=input("Enter the string: ").lower()
c=0
for i in s:
    if i in 'aeiou':
        c=c+1
print("total vowels =",c)
```

Output:

```
Enter the string: education
total vowels = 5
```

Program Logic:

- Take string input from the user.
- Convert string into lowercase.
- Check whether character is vowel or not.
- Increase counter for every vowel.
- Print total vowel count.

Mathematical Logic:

- Vowels are a, e, i, o, u.
- Counting operation gives total frequency.

24. Count Total Number of Consonants in the Given String

Program:

```
s=input("Enter the string: ").lower()
c=0
for i in s:
    if i.isalpha() and i not in 'aeiou':
        c=c+1
print("total consonants =",c)
```

Output:

```
Enter the string: education
```


total consonants = 4

Program Logic:

- Take string input from the user.
- Check alphabet characters only.
- Ignore vowels and count remaining letters.
- Print total consonant count.

Mathematical Logic:

- Consonants are alphabets except vowels.

25. Find the First Non-Repeated Character in the Given String

Program:

```
s=input("Enter the string: ")
for i in s:
    if s.count(i)==1:
        print(i)
        break
```

Output:

```
Enter the string: aabbccdddef
e
```

Program Logic:

- Take string input from the user.
- Traverse each character one by one.
- Check frequency using count().
- Print first character having count 1.

Mathematical Logic:

- Non-repeated character occurs exactly one time.

26. Replace Spaces with Hyphen in the Given String

Program:

```
s=input("Enter the string: ")
print(s.replace(" ","-"))
```

Output:

```
Enter the string: python is easy
python-is-easy
```

Program Logic:

- Take string input from the user.
- Replace every space with hyphen.
- Print updated string.

Mathematical Logic:

- Replacement substitutes one symbol with another symbol.

27. Count Uppercase and Lowercase Characters

Program:

```
s=input("Enter the string: ")
u=0
l=0
for i in s:
    if i.isupper():
```

```

        u=u+1
    elif i.islower():
        l=l+1
    print("uppercase =",u)
    print("lowercase =",l)

```

Output:

```

Enter the string: PyThOn
uppercase = 3
lowercase = 3

```

Program Logic:

- Take string input from the user.
- Check uppercase characters using isupper().
- Check lowercase characters using islower().
- Count both separately.
- Print results.

Mathematical Logic:

- Uppercase and lowercase letters belong to different ASCII ranges.

28. Remove All Spaces from the Given String

Program:

```

s=input("Enter the string: ")
print(s.replace(" ",""))

```

Output:

```

Enter the string: python programming
pythonprogramming

```

Program Logic:

- Take string input from the user.
- Replace spaces with empty string.
- Print updated string.

Mathematical Logic:

- Empty string removes unwanted spaces from sequence.

29. Convert First Character of Each Word into Uppercase

Program:

```

s=input("Enter the string: ")
l=s.split()
r=[]
for i in l:
    r.append(i.capitalize())
print(" ".join(r))

```

Output:

```

Enter the string: python is easy
Python Is Easy

```

Program Logic:

- Take string input from the user.
- Split string into words.
- Capitalize first letter of every word.
- Join all words into sentence.
- Print final string.

Mathematical Logic:

- Capitalization converts first alphabet into uppercase.

30. Find the Longest Word in the Given Sentence**Program:**

```
s=input("Enter the sentence: ")
l=s.split()
m=l[0]
for i in l:
    if len(i)>len(m):
        m=i
print("longest word =",m)
```

Output:

```
Enter the sentence: python programming language
longest word = programming
```

Program Logic:

- Take sentence input from the user.
- Split sentence into words.
- Compare lengths of all words.
- Store longest word.
- Print longest word.

Mathematical Logic:

- Word having maximum length is longest word.

31. Find the Smallest Word in the Given Sentence**Program:**

```
s=input("Enter the sentence: ")
l=s.split()
m=l[0]
for i in l:
    if len(i)<len(m):
        m=i
print("smallest word =",m)
```

Output:

```
Enter the sentence: python is easy
smallest word = is
```

Program Logic:

- Take sentence input from the user.
- Split sentence into words.
- Compare lengths of all words.
- Store smallest word.
- Print smallest word.

Mathematical Logic:

- Word having minimum length is smallest word.

32. Check Whether Two Strings are Rotations of Each Other**Program:**

```
s1=input("Enter first string: ")
```

```
s2=input("Enter second string: ")
if len(s1)==len(s2) and s2 in (s1+s1):
    print("rotation")
else:
    print("not rotation")
```

Output:

```
Enter first string: ABCD
Enter second string: CDAB
rotation
```

Program Logic:

- Take two strings from the user.
- Check equal length of both strings.
- Concatenate first string with itself.
- Check whether second string exists in new string.
- Print result.

Mathematical Logic:

- Rotation preserves character order in circular form.

33. Find Duplicate Characters in the Given String

Program:

```
s=input("Enter the string: ")
for i in set(s):
    if s.count(i)>1:
        print(i)
```

Output:

```
Enter the string: programming
r
g
m
```

Program Logic:

- Take string input from the user.
- Remove duplicates using set().
- Check frequency of each character.
- Print characters having count more than 1.

Mathematical Logic:

- Duplicate characters repeat multiple times.

34. Convert Lowercase Characters into Uppercase

Program:

```
s=input("Enter the string: ")
print(s.upper())
```

Output:

```
Enter the string: python
PYTHON
```

Program Logic:

- Take string input from the user.
- Convert all lowercase letters into uppercase.
- Print updated string.

Mathematical Logic:

- Uppercase letters have different ASCII values from lowercase letters.

35. Convert Uppercase Characters into Lowercase

Program:

```
s=input("Enter the string: ")
print(s.lower())
```

Output:

```
Enter the string: PYTHON
python
```

Program Logic:

- Take string input from the user.
- Convert all uppercase letters into lowercase.
- Print updated string.

Mathematical Logic:

- Lowercase letters belong to another ASCII range.

36. Check Whether String Contains Only Digits

Program:

```
s=input("Enter the string: ")
if s.isdigit():
    print("contains only digits")
else:
    print("not only digits")
```

Output:

```
Enter the string: 12345
contains only digits
```

Program Logic:

- Take string input from the user.
- Use isdigit() function to check digits.
- Print result based on condition.

Mathematical Logic:

- Digits range from 0 to 9 only.

37. Find ASCII Value of Each Character in the String

Program:

```
s=input("Enter the string: ")
for i in s:
    print(i,"=",ord(i))
```

Output:

```
Enter the string: ABC
A = 65
B = 66
C = 67
```

Program Logic:

- Take string input from the user.
- Traverse each character.

- Convert character into ASCII value using ord().
- Print character with ASCII value.

Mathematical Logic:

- ASCII assigns unique numeric value to every character.

38. Count Total Number of Words in the Given Sentence

Program:

```
s=input("Enter the sentence: ")
l=s.split()
print("total words =",len(l))
```

Output:

```
Enter the sentence: python is very easy
total words = 4
```

Program Logic:

- Take sentence input from the user.
- Split sentence into words.
- Count total words using len().
- Print total number of words.

Mathematical Logic:

- Number of separated words gives word count.

39. Find Total Number of Digits Present in the String

Program:

```
s=input("Enter the string: ")
c=0
for i in s:
    if i.isdigit():
        c=c+1
print("total digits =",c)
```

Output:

```
Enter the string: abc123de45
total digits = 5
```

Program Logic:

- Take string input from the user.
- Traverse each character.
- Check whether character is digit.
- Increase counter for every digit.
- Print total digit count.

Mathematical Logic:

- Digits range from 0 to 9.

40. Find Total Number of Special Characters in the String

Program:

```
s=input("Enter the string: ")
c=0
for i in s:
    if not i.isalnum() and i!=" ":
        c=c+1
print("special characters =",c)
```

Output:

Enter the string: abc@12#&
special characters = 3

Program Logic:

- Take string input from the user.
- Check characters which are not alphabet or digit.
- Ignore spaces.
- Count special symbols.
- Print total count.

Mathematical Logic:

- Special characters are symbols other than alphabets and numbers.

41. Remove All Vowels from the Given String**Program:**

```
s=input("Enter the string: ")
r=""

for i in s:
    if i.lower() not in 'aeiou':
        r=r+i
print(r)
```

Output:

Enter the string: education
dctn

Program Logic:

- Take string input from the user.
- Traverse each character.
- Ignore vowels and store remaining characters.
- Print final string.

Mathematical Logic:

- Vowels are removed from the sequence.

42. Check Whether the String Starts with a Vowel**Program:**

```
s=input("Enter the string: ").lower()

if s[0] in 'aeiou':
    print("starts with vowel")
else:
    print("does not start with vowel")
```

Output:

Enter the string: apple
starts with vowel

Program Logic:

- Take string input from the user.
- Convert string into lowercase.
- Check first character.
- Print result based on condition.

Mathematical Logic:

- First character determines vowel condition.

43. Print Each Character with Its Position

Program:

```
s=input("Enter the string: ")  
  
for i in range(len(s)):  
    print(s[i],"=",i)
```

Output:

Enter the string: python

p = 0

y = 1

t = 2

h = 3

o = 4

n = 5

Program Logic:

- Take string input from the user.
- Traverse string using index values.
- Print character with its index position.

Mathematical Logic:

- Indexing starts from 0 in Python.

44. Find the Frequency of Vowels in the Given String

Program:

```
s=input("Enter the string: ").lower()  
for i in 'aeiou':  
    print(i,"=",s.count(i))
```

Output:

Enter the string: education

a = 1

e = 1

i = 1

o = 1

u = 1

Program Logic:

- Take string input from the user.
- Convert string into lowercase.
- Count frequency of every vowel.
- Print vowel frequencies separately.

Mathematical Logic:

- Frequency represents total occurrence of vowels.

45. Reverse Only Words Having Even Length

Program:

```
s=input("Enter the sentence: ")  
l=s.split()
```



```

for i in range(len(l)):
    if len(l[i])%2==0:
        l[i]=l[i][::-1]
print(" ".join(l))

```

Output:

Enter the sentence: one four six seven
 one ruof xis seven

Program Logic:

- Take sentence input from the user.
- Split sentence into words.
- Check length of each word.
- Reverse only even length words.
- Print final sentence.

Mathematical Logic:

- Modulus operator identifies even length words.

46. Find the Highest Frequency Character in the String

Program:

```

s=input("Enter the string: ")
m=0
ch=""
for i in set(s):

    if s.count(i)>m:
        m=s.count(i)
        ch=i
print("highest frequency character =",ch)

```

Output:

Enter the string: aaabbbbccdd
 highest frequency character = b

Program Logic:

- Take string input from the user.
- Traverse unique characters.
- Compare frequencies of characters.
- Store highest occurring character.
- Print result.

Mathematical Logic:

- Maximum frequency gives most repeated character.

47. Replace All Digits in the String with '*'

Program:

```

s=input("Enter the string: ")
r=""
for i in s:
    if i.isdigit():
        r=r+'*'
    else:

```

```
    r=r+i
print(r)
```

Output:

```
Enter the string: abc123de45
abc***de**
```

Program Logic:

- Take string input from the user.
- Check every character.
- Replace digits with '*'.
- Keep remaining characters unchanged.
- Print updated string.

Mathematical Logic:

- Replacement operation substitutes digits with symbols.

48. Find the Last Non-Repeated Character in the Given String

Program:

```
s=input("Enter the string: ")

for i in s[::-1]:
    if s.count(i)==1:
        print(i)
        break
```

Output:

```
Enter the string: aabbccdddefg
g
```

Program Logic:

- Take string input from the user.
- Traverse the string from reverse direction.
- Check frequency of each character.
- Print first non-repeated character from end.

Mathematical Logic:

- Character occurring exactly one time is non-repeated.

49. Check Whether Two Strings are Equal Ignoring Case

Program:

```
s1=input("Enter first string: ")
s2=input("Enter second string: ")
if s1.lower()==s2.lower():
    print("equal")
else:
    print("not equal")
```

Output:

```
Enter first string: Python
Enter second string: PYTHON
```

equal

Program Logic:

- Take two strings from the user.
- Convert both strings into lowercase.

- Compare both strings.
- Print result based on condition.

Mathematical Logic:

- Case conversion removes uppercase and lowercase difference.

50. Find the Percentage of Uppercase, Lowercase, Digits, and Special Characters

Program:

```
s=input("Enter the string: ")
u=l=d=sp=0
for i in s:
    if i.isupper():
        u=u+1
    elif i.islower():
        l=l+1
    elif i.isdigit():
        d=d+1
    else:
        sp=sp+1
n=len(s)
print("uppercase percentage =",u*100/n)
print("lowercase percentage =",l*100/n)
print("digit percentage =",d*100/n)
print("special character percentage =",sp*100/n)
```

Output:

```
Enter the string: PyTh12@#
uppercase percentage = 25.0
lowercase percentage = 25.0
digit percentage = 25.0
special character percentage = 25.0
```

Program Logic:

- Take string input from the user.
- Count uppercase, lowercase, digits, and special symbols.
- Find total length of string.
- Calculate percentage of each category.
- Print all percentages.

Mathematical Logic:

Percentage= $\frac{\text{Category Count} \times 100}{\text{Total Characters}}$

- Percentage formula calculates contribution of each character type.

1. Add Two Numbers

Program:

```
def add(a,b):  
    return a+b  
x=int(input("Enter first number: "))  
y=int(input("Enter second number: "))  
print("sum =",add(x,y))
```

Output:

```
Enter first number: 5  
Enter second number: 3  
sum = 8
```

Program Logic:

- Define a function to add two numbers.
- Take two numbers as input from the user.
- Call the function and print the result.

Mathematical Logic:

```
[  
Sum = a+b  
]
```

- Addition combines two numbers into one value.

2. Square of a Number

Program:

```
def square(n):  
    return n*n  
  
x=int(input("Enter a number: "))  
  
print("square =",square(x))
```

Output:

```
Enter a number: 4  
square = 16
```

Program Logic:

- Define a function to calculate square.
- Take a number as input.
- Multiply the number by itself.
- Print the square value.

Mathematical Logic:

```
[  
Square=n^2  
]
```

- Square means multiplying a number by itself.

3. Even or Odd

Program:

```
def check(n):  
    if n%2==0:  
        return "Even"
```

```

else:
    return "Odd"
x=int(input("Enter a number: "))

```

```

print(check(x))

```

Output:

```

Enter a number: 7
Odd

```

Program Logic:

- Define a function to check even or odd.
- Divide the number by 2 using modulus operator.
- If remainder is 0, number is even otherwise odd.

Mathematical Logic:

```

[
n % 2 = 0
]

```

- Even numbers are divisible by 2 completely.

4. Maximum of Three Numbers

Program:

```

def maximum(a,b,c):
    if a>b and a>c:
        return a
    elif b>c:
        return b
    else:
        return c
x=int(input("Enter first number: "))
y=int(input("Enter second number: "))
z=int(input("Enter third number: "))
print("maximum =",maximum(x,y,z))

```

Output:

```

Enter first number: 10
Enter second number: 25
Enter third number: 15
maximum = 25

```

Program Logic:

- Define a function to compare three numbers.
- Use if-elif-else conditions.
- Return the largest number.

Mathematical Logic:

```

[
Maximum=max(a,b,c)
]

```

- Maximum means the greatest value among numbers.

5. Factorial of a Number

Program:

```
def factorial(n):
    f=1
    for i in range(1,n+1):
        f=f*i
    return f
x=int(input("Enter a number: "))
print("factorial =",factorial(x))
```

Output:

```
Enter a number: 5
factorial = 120
```

Program Logic:

- Define a factorial function.
- Start multiplication from 1 to n.
- Store multiplication result in variable f.
- Print factorial value.

Mathematical Logic:

[
 $n! = 1 \times 2 \times 3 \times \dots \times n$
]

- Factorial is the product of all positive integers up to n.

6. Multiplication Table

Program:

```
def table(n):
    for i in range(1,11):
        print(n,"x",i,"=",n*i)
x=int(input("Enter a number: "))
table(x)
```

Output:

```
Enter a number: 3
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27
3 x 10 = 30
```

Program Logic:

- Define a function for multiplication table.
- Use loop from 1 to 10.
- Multiply the number with loop variable.
- Print the table.

Mathematical Logic:

[

```
Table=n\times i  
]
```

- Multiplication table shows repeated multiplication.

7. Area of a Circle

Program:

```
def area(r):  
    return 3.14*r*r  
x=float(input("Enter radius: "))  
print("area =",area(x))
```

Output:

```
Enter radius: 7  
area = 153.86
```

Program Logic:

- Define a function to calculate circle area.
- Take radius as input.
- Use formula πr^2 .
- Print the area.

Mathematical Logic:

```
[  
Area=\pi r^2  
]
```

- Area of circle depends on radius.

8. Sum from 1 to n

Program:

```
def sum_n(n):  
    s=0  
    for i in range(1,n+1):  
        s=s+i  
    return s  
x=int(input("Enter a number: "))  
print("sum =",sum_n(x))
```

Output:

```
Enter a number: 10  
sum = 55
```

Program Logic:

- Define a function to calculate sum.
- Use loop from 1 to n.
- Add each number into variable s.
- Print total sum.

Mathematical Logic:

```
[  
Sum=\frac{n(n+1)}{2}  
]
```

- Formula gives sum of first n natural numbers.

9. Average of Three Numbers

Program:

```
def average(a,b,c):  
    return (a+b+c)/3  
x=int(input("Enter first number: "))  
y=int(input("Enter second number: "))  
z=int(input("Enter third number: "))  
print("average =",average(x,y,z))
```

Output:

```
Enter first number: 10  
Enter second number: 20  
Enter third number: 30  
average = 20.0
```

Program Logic:

- Define a function to calculate average.
- Add all three numbers.
- Divide the sum by 3.
- Print average value.

Mathematical Logic:

```
[  
Average=\frac{a+b+c}{3}  
]
```

- Average means equal distribution of values.

10. Cube of a Number

Program:

```
def cube(n):  
    return n*n*n  
x=int(input("Enter a number: "))  
print("cube =",cube(x))
```

Output:

```
Enter a number: 3  
cube = 27
```

Program Logic:

- Define a function to calculate cube.
- Multiply the number three times.
- Print cube value.

Mathematical Logic:

```
[  
Cube=n^3  
]
```

- Cube means multiplying a number by itself three times.

11. Check if a Number is Prime

Program:

```
def prime(n):
    if n<=1:
        return "Not Prime"
    for i in range(2,n):
        if n%i==0:
            return "Not Prime"
    return "Prime"
x=int(input("Enter a number: "))
print(prime(x))
```

Output:

Enter a number: 7

Prime

Program Logic:

- Define a function to check prime number.
- If number is less than or equal to 1, it is not prime.
- Check divisibility from 2 to n-1.
- If divisible, number is not prime otherwise prime.

Mathematical Logic:

```
[
n % i = 0
]
```

- Prime numbers are divisible only by 1 and itself.

12. Sum of Digits of a Number

Program:

```
def sum_digit(n):
    s=0
    while n>0:
        r=n%10
        s=s+r
        n=n//10
    return s
x=int(input("Enter a number: "))
print("sum of digits =",sum_digit(x))
```

Output:

Enter a number: 123

sum of digits = 6

Program Logic:

- Define a function to calculate sum of digits.
- Extract last digit using modulus operator.
- Add digits into sum variable.
- Remove last digit using floor division.

Mathematical Logic:

```
[
Digit=n%10
]
```

- Digits are extracted one by one from the number.

13. Reverse of a Number

Program:

```
def reverse(n):
    rev=0
    while n>0:
        r=n%10
        rev=rev*10+r
        n=n//10
    return rev
x=int(input("Enter a number: "))
print("reverse =",reverse(x))
```

Output:

```
Enter a number: 12345
reverse = 54321
```

Program Logic:

- Define a function to reverse number.
- Extract digits one by one.
- Add digits in reverse order.
- Print reversed number.

Mathematical Logic:

```
[
Reverse=(Reverse\times10)+Digit
]
```

- Digits are rearranged from last to first.

14. Check if a Number is Palindrome

Program:

```
def palindrome(n):
    temp=n
    rev=0
    while n>0:
        r=n%10
        rev=rev*10+r
        n=n//10
    if temp==rev:
        return "Palindrome"
    else:
        return "Not Palindrome"
x=int(input("Enter a number: "))
print(palindrome(x))
```

Output:

```
Enter a number: 121
Palindrome
```

Program Logic:

- Define a function to check palindrome.
- Reverse the number.

- Compare original and reversed numbers.
- If both are same, it is palindrome.

Mathematical Logic:

```
[
Original=Reverse
]
```

- Palindrome numbers remain same after reversing.

15. Find the Greatest Common Divisor (GCD)

Program:

```
def gcd(a,b):
    while b!=0:
        a,b=b,a%b
    return a
x=int(input("Enter first number: "))
y=int(input("Enter second number: "))
print("gcd =",gcd(x,y))
```

Output:

```
Enter first number: 56
Enter second number: 98
gcd = 14
```

Program Logic:

- Define a function to calculate GCD.
- Use Euclidean algorithm.
- Replace numbers repeatedly until remainder becomes 0.
- Final value of a is GCD.

Mathematical Logic:

```
[
GCD(a,b)=GCD(b,a%b)
]
```

- GCD is the greatest number dividing both numbers completely.

16. Count the Number of Digits in a Number

Program:

```
def count_digit(n):
    c=0
    while n>0:
        c=c+1
        n=n//10
    return c
x=int(input("Enter a number: "))
print("total digits =",count_digit(x))
```

Output:

```
Enter a number: 12345
total digits = 5
```

Program Logic:

- Define a function to count digits.
- Remove digits one by one using floor division.

- Increase counter each time.
- Print total count.

Mathematical Logic:

```
[
n=n//10
]
```

- Each division removes one digit from the number.

17. Sum of Even Numbers in a Given Range

Program:

```
def even_sum(start,end):
    s=0
    for i in range(start,end+1):
        if i%2==0:
            s=s+i
    return s
a=int(input("Enter starting number: "))
b=int(input("Enter ending number: "))
print("sum of even numbers =",even_sum(a,b))
```

Output:

```
Enter starting number: 1
Enter ending number: 10
sum of even numbers = 30
```

Program Logic:

- Define a function to find sum of even numbers.
- Use loop from start to end.
- Check even numbers using modulus operator.
- Add even numbers into sum variable.

Mathematical Logic:

```
[
n%2=0
]
```

- Even numbers are divisible by 2.

18. Print All Prime Numbers Between 1 and n

Program:

```
def prime_numbers(n):
    for i in range(2,n+1):
        flag=0
        for j in range(2,i):
            if i%j==0:
                flag=1
                break
        if flag==0:
            print(i,end=" ")
x=int(input("Enter a number: "))
prime_numbers(x)
```

Output:

Enter a number: 10

2 3 5 7

Program Logic:

- Define a function to print prime numbers.
- Check each number from 2 to n.
- If number has no divisor, print it.
- Continue until n.

Mathematical Logic:

[
Prime\ Numbers=Numbers\ having\ only\ two\ factors
]

- Prime numbers are divisible only by 1 and itself.

19. Convert Celsius to Fahrenheit

Program:

```
def fahrenheit(c):  
    return (c*9/5)+32  
x=float(input("Enter temperature in Celsius: "))  
print("fahrenheit =",fahrenheit(x))
```

Output:

```
Enter temperature in Celsius: 25  
fahrenheit = 77.0
```

Program Logic:

- Define a function to convert Celsius into Fahrenheit.
- Use conversion formula.
- Print Fahrenheit temperature.

Mathematical Logic:

[
 $F=(C \times \frac{9}{5})+32$
]

- Formula converts Celsius temperature into Fahrenheit.

20. Calculate Percentage from Marks of 5 Subjects

Program:

```
def percentage(a,b,c,d,e):  
    total=a+b+c+d+e  
    per=total/5  
    return per  
m1=int(input("Enter marks of subject 1: "))  
m2=int(input("Enter marks of subject 2: "))  
m3=int(input("Enter marks of subject 3: "))  
m4=int(input("Enter marks of subject 4: "))  
m5=int(input("Enter marks of subject 5: "))  
print("percentage =",percentage(m1,m2,m3,m4,m5))
```

Output:

```
Enter marks of subject 1: 80  
Enter marks of subject 2: 90  
Enter marks of subject 3: 85
```

Enter marks of subject 4: 70

Enter marks of subject 5: 95

percentage = 84.0

Program Logic:

- Define a function to calculate percentage.
- Add marks of all subjects.
- Divide total marks by 5.
- Print percentage.

Mathematical Logic:

```
[  
Percentage=\frac{Total\ Marks}{Number\ of\ Subjects}  
]
```

- Percentage shows average marks obtained.

21. Nth Fibonacci Number using Recursion

Program:

```
def fibonacci(n):  
    if n<=1:  
        return n  
    return fibonacci(n-1)+fibonacci(n-2)  
x=int(input("Enter position: "))  
print("fibonacci number =",fibonacci(x))
```

Output:

Enter position: 6

fibonacci number = 8

Program Logic:

- Define a recursive function for Fibonacci series.
- If n is 0 or 1, return n.
- Otherwise return sum of previous two Fibonacci numbers.
- Print nth Fibonacci number.

Mathematical Logic:

```
[  
F(n)=F(n-1)+F(n-2)  
]
```

- Fibonacci series is formed by adding previous two numbers.

22. Check if a Number is an Armstrong Number

Program:

```
def armstrong(n):  
    temp=n  
    s=0  
    digits=len(str(n))  
    while n>0:  
        r=n%10  
        s=s+(r**digits)  
        n=n//10  
    if temp==s:  
        return "Armstrong Number"
```

```

else:
    return "Not Armstrong Number"
x=int(input("Enter a number: "))
print(armstrong(x))

```

Output:

Enter a number: 153
Armstrong Number

Program Logic:

- Define a function to check Armstrong number.
- Count total digits in number.
- Raise each digit to power of total digits.
- Add all powered values.
- Compare result with original number.

Mathematical Logic:

```

[
153=1^3+5^3+3^3
]

```

- Armstrong number is equal to sum of powered digits.

23. Return All Factors of a Number

Program:

```

def factors(n):
    for i in range(1,n+1):
        if n%i==0:
            print(i,end=" ")
x=int(input("Enter a number: "))
factors(x)

```

Output:

Enter a number: 12
1 2 3 4 6 12

Program Logic:

- Define a function to find factors.
- Check divisibility from 1 to n.
- If divisible, print the factor.

Mathematical Logic:

```

[
n%i=0
]

```

- Factors divide a number completely without remainder.

24. Check if a Number is Perfect Number

Program:

```

def perfect(n):
    s=0
    for i in range(1,n):
        if n%i==0:
            s=s+i

```

```

if s==n:
    return "Perfect Number"
else:
    return "Not Perfect Number"
x=int(input("Enter a number: "))
print(perfect(x))

```

Output:

Enter a number: 28
Perfect Number

Program Logic:

- Define a function to check perfect number.
- Find proper divisors of number.
- Add all divisors.
- If sum equals original number, it is perfect.

Mathematical Logic:

```

[
28=1+2+4+7+14
]

```

- Perfect number equals sum of proper divisors.

25. Find Least Common Multiple (LCM)

Program:

```

def gcd(a,b):
    while b!=0:
        a,b=b,a%b
    return a
def lcm(a,b):
    return (a*b)//gcd(a,b)
x=int(input("Enter first number: "))
y=int(input("Enter second number: "))
print("lcm =",lcm(x,y))

```

Output:

Enter first number: 12
Enter second number: 18
lcm = 36

Program Logic:

- Define function for GCD.
- Use formula to calculate LCM.
- Divide product of numbers by GCD.
- Print LCM value.

Mathematical Logic:

```

[
 $LCM = \frac{a \times b}{GCD(a,b)}$ 
]

```

- LCM is the smallest common multiple of two numbers.

26. Find Second Largest Number in a List

Program:


```
def second_largest(l):

    l=list(set(l))

    l.sort()

    return l[-2]

numbers=[1,4,7,4,9,7]

print("second largest =",second_largest(numbers))
```

Output:

second largest = 7

Program Logic:

- Define a function to find second largest number.
- Remove duplicate elements using set().
- Sort the list in ascending order.
- Return second last element.

Mathematical Logic:

```
[
Second\ Largest = Sorted[-2]
]
```

- Second largest is the value just smaller than maximum.

27. Sum of All Prime Numbers Between 1 and n

Program:

```
def prime(n):
    if n<=1:
        return False
    for i in range(2,n):
        if n%i==0:
            return False
    return True
def sum_prime(n):
    s=0
    for i in range(2,n+1):
        if prime(i):
            s=s+i
    return s
x=int(input("Enter a number: "))
print("sum of prime numbers =",sum_prime(x))
```

Output:

Enter a number: 10

sum of prime numbers = 17

Program Logic:

- Define a function to check prime number.
- Define another function to calculate sum of primes.

- Add all prime numbers between 1 and n.
- Print total sum.

Mathematical Logic:

```
[
2+3+5+7=17
]
```

- Prime numbers are added together to get total sum.

28. Check if a Number is a Harshad Number

Program:

```
def harshad(n):
    temp=n
    s=0
    while n>0:
        r=n%10
        s=s+r
        n=n//10
    if temp%s==0:
        return "Harshad Number"
    else:
        return "Not Harshad Number"
x=int(input("Enter a number: "))
print(harshad(x))
```

Output:

Enter a number: 18

Harshad Number

Program Logic:

- Define a function to check Harshad number.
- Find sum of digits.
- Divide original number by digit sum.
- If divisible, it is Harshad number.

Mathematical Logic:

```
[
Number%Sum\ of\ Digits=0
]
```

- Harshad numbers are divisible by sum of digits.

29. Digital Root

Program:

```
def digital_root(n):
    while n>9:
        s=0
        while n>0:
            r=n%10
            s=s+r
            n=n//10
        n=s
```

```

    return n
x=int(input("Enter a number: "))
print("digital root =",digital_root(x))

```

Output:

Enter a number: 987
digital root = 6

Program Logic:

- Define a function to calculate digital root.
- Find sum of digits repeatedly.
- Continue until single digit remains.
- Print final digit.

Mathematical Logic:

```

[
987 \rightarrow 9+8+7=24 \rightarrow 2+4=6
]

```

- Digital root is obtained by repeated digit summation.

30. Trailing Zeros in Factorial

Program:

```

def trailing_zero(n):
    c=0
    while n>=5:
        n=n//5
        c=c+n
    return c
x=int(input("Enter a number: "))
print("trailing zeros =",trailing_zero(x))

```

Output:

Enter a number: 100
trailing zeros = 24

Program Logic:

- Define a function to count trailing zeros in factorial.
- Divide number repeatedly by 5.
- Add quotient values into counter.
- Print total trailing zeros.

Mathematical Logic:

```

[
Zeros=\left(\frac{n}{5}\right)+\left(\frac{n}{25}\right)+\left(\frac{n}{125}\right)
]

```

- Trailing zeros depend on number of factors of 5 in factorial.